HOTEL MANAGEMENT SYSTEM

A Mini Project Report Submitted to the University of Madras, in partial fulfillment of the requirements for the award of the degree

BACHELOR OF SCIENCE

Submitted by	Register No.
ASHWINI M	222204606
PRIYADHARSHINI B	222204648
ARSHEEN PARVEEN R	222204604

UNDER THE GUIDANCE OF

Mrs. H.SHADHIKA M.C.A., M.Phil., B.Ed.,

Assistant Professor

DEPARTMENT OF COMPUTER SCIENCE



VIDHYA SAGAR WOMEN'S COLLEGE

VEDHANARAYANAPURAM

CHENGALPATTU – 603111

APRIL-2025

vidhya sagar
WOMEN'S COLLEGE
Affiliated to University of Madras
Reaccredited by NAAC
Chengalpattu - 603 111

BONAFIDE CERTIFICATE

Certified that this report titled "**HOTEL MANAGEMENT SYSTEM**" is a Bonafide record of the project work done by **ASHWINI M (222204606), PRIYADHARSHINI B (222204648), ARSHEEN PARVEEN R (222204604)** under our supervision and guidance, towards partial fulfillment of the requirement for award of the Degree of **B.Sc., Computer Science** of Vidhya Sagar Women's College for the academic year 2024-2025.

Signature of the Project Guide **Signature of the Head**

Submitted to the University Viva-voce held on ……. at Vidhya Sagar Women's College, Chenalpatgtu-603111.

Internal Examiner **External Examiner**

ACKNOWLEDGEMENT

First of all, We would like to express our heartfelt thanks to **Shri. Vikas Surana**, Correspondent, Vidhya Sagar Women's College, Chengalpattu, for his direct and indirect support and encouragement.

We would like to express our sincere thanks to our Principal **Dr.R.Arunadevi M.C.A., M.Phil., P.hD.,** Vidhya Sagar Women's College, Chengalpattu, for her direct and indirect support and encouragement.

We Wish to express our grateful thanks to **Mr.P.D.Yekanath M.Sc., M.Phil., SET,** Head, Department of Computer Science, Vidhya Sagar Women's college, Chengalpattu, for the encouragement in successful completion of our project.

We wholeheartedly extent our gratefulness to **Mr. R. Samuel Kirubakaran, M.Sc., M.C.A., M.Phil., B.Ed.,** Deputy Head, Department of B.Sc., Computer Science, Vidhya Sagar Women's College Chengalpattu, who has been helping me a lot for the completion of this project.

We extend our sincere and heartfelt thanks to our esteemed guide **Mrs. H.Shadhika M.C.A., M.Phil., B.Ed.,** Assistant Professor, Department of Computer Science, Vidhya Sagar Women's College, Chengalpattu, for providing us the right guidance and advice at the crucial junctures and for showing us the right way .

We also extend our gratefulness to all the staff members of the Department of Computer Science for their support during the course of my study.

We also take this occasion to thank God, almighty for blessing us with his grace and taking our endeavour to a successful culmination.

We would like to thank our friends and family for the support and encouragement they have given us during the course of our work.

TABLE OF CONTENT

S.NO	CONTENT	PAGE NO
1	**ABSTRACT**	01
2	**INTRODUCTION**	02
3	**SYSTEM STUDY** 3.1FEASIBILITY STUDY 3.2ECONOMICAL FEASIBILITY 3.3TECHNICAL FEASIBILITY 3.4 SOCIAL FEASIBILITY	03
4	**PROJECT DESCRIPTION** 4.1 MODULES DESCRIPTION 4.2 TRAINING MODULE 4.3 OBJECT DETECTION	04
5	**SYSTEM SPECIFICATION** 5.1 HARDWARE SPECIFICATION 5.2 SOFTWARE SPECIFICATION	05
6	**SOFTWARE DESCRIPTION** 6.1 STRUCTURE OF ANDROID 6.2 APPLICATION FRAMEWORK 6.3 LIBRARIES 6.4 ANDROID RUN TIMES 6.5 LINUX KERNEL 6.6 FEATURES OF ANDROID 6.7 CATEGORIES OF ANDROID APPLICATION	06
7	**SYSTEM DESIGN** 7.1 USECASE DIAGRAM 7.2 CLASS DIAGRAM 7.3 SEQUENCE DIAGRAM 7.4 ACTIVITY DIAGRAM	07
8	**SYSTEM TESTING** 8.1 TESTS 8.2 TYPE OF TESTING 8.3 UNIT TESTING 8.4 INTEGRATION TESTING 8.5 FUNCTIONAL TESTING 8.6 SYSTEM TESTING 8.7 WHITE BOX TESTING	08
9	**SOURCE CODE**	09
10	**SCREEN LAYOUTS**	102
11	**FUTURE ENHANCEMENT**	105
12	**CONCLUSIONS**	106
13	**BIBILIOGRAPHY**	107

ABSTRACT

1. ABSTRACT

The Hotel Management System is a comprehensive web based application designed to streamline hotel operations, enhance guest experiences, and improve overall efficiency through automated booking processes and administrative workflows. Built with Express.js and Mongo DB on the back end, and React.js with Tailwind CSS on the frontend, the system implements a modern micro services architecture following REST API principles.

Key features are room management and allocation, online reservation and booking system, guest information management, service management, billing and payment management , reporting and analytics.

Benefits are improved operational efficiency, enhanced guest experiences, reduced errors and paperwork, increased revenue through optimized room allocation.

INTRODUCTION

2. INTRODUCTION

The Hotel Management application is designed to streamline hotel operations, including room bookings, user management, and administrative tasks. It provides a comprehensive platform for managing hotel resources efficiently.

Welcome to Hotel Management App – Your Ultimate Hotel Booking Companion!

Planning a getaway or business trip? [**Hotel Management App**] makes booking your perfect hotel a breeze. With our user-friendly interface, you can easily explore a wide selection of hotels, from luxurious resorts to budget-friendly options, all in one place. Whether you're traveling for work, relaxation, or adventure, we ensure a smooth, hassle-free booking experience.

SYSTEM STUDY

3. SYSTEM STUDY

3.1 FEASIBILITY STUDY

The project has been evaluated for its feasibility in terms of technical implementation, economic value, and social impact. The study confirms that the Hotel Management application is viable to develop and deploy.

3.2 ECONOMICAL FEASIBILITY

The application provides economic value by optimizing hotel operations and improving customer service. The development and maintenance costs are justified by the increased efficiency and customer satisfaction.

3.3 TECHNICAL FEASIBILITY

The application is built using a modern web stack with a Node.js backend and a React frontend. The technical stack is well-supported and has active communities, making development and maintenance feasible.

3.4 SOCIAL FEASIBILITY

The application addresses a significant need for efficient hotel management. By providing tools for managing bookings and resources, it helps improve service delivery and customer experience.

PROJECT DESCRIPTION

4. PROJECT DESCRIPTION

4.1 MODULES DESCRIPTION

Core Modules:

1. Backend Module: Manages API endpoints, database interactions, and authentication.

2. Frontend Module: Provides the user interface for hotel staff and customers.

Supporting Modules:

1. Middleware Module: Handles authentication and request processing.

2. Utility Module: Provides helper functions for the backend.

4.2 TRAINING MODULE

Not applicable for this application.

4.3 OBJECT DETECTION

Not applicable for this application.

SYSTEM SPECIFICATION

5. SYSTEM SPECIFICATION

5.1 HARDWARE SPECIFICATION

- Client-side: Any modern device with a web browser.

- Server-side: Node.js server environment.

5.2 SOFTWARE SPECIFICATION

- Backend: Node.js, Express.js

-Frontend: React, Tailwind CSS

-Database: MongoDB

-Environment: Vercel for deployment

SOFTWARE DESCRIPTION

6. SOFTWARE DESCRIPTION

6.1 STRUCTURE OF APPLICATION

The application follows a client-server architecture with a RESTful API backend and a React-based frontend.

6.2 APPLICATION FRAMEWORK

The backend is built using Node.js and Express.js, while the frontend uses React for building user interfaces.

6.3 LIBRARIES

The application leverages various libraries including Express for server-side logic, Mongoose for MongoDB interactions, and React for the frontend.

6.4 RUNTIME

The application runs on a Node.js server and is deployed using Vercel.

6.5 BROWSER COMPATIBILITY

The frontend is designed to work on modern browsers that support ES6+ features.

6.6 FEATURES OF THE APPLICATION

- Room booking and management

- User authentication and authorization

- Dashboard for hotel staff and administrators

- Contact and information pages

6.7 CATEGORIES OF APPLICATION COMPONENTS

- Core Components: Backend API, Frontend UI

- Utility Components: Middleware, Helper Functions

SYSTEM DESIGN

7. SYSTEM DESIGN

7.1 USE CASE DIAGRAM

The application supports the following use cases:

- User registers and logs in

- Customer books a room

- Administrator manages bookings and users

7.2 CLASS DIAGRAM

The application uses JavaScript classes and modules for data management and processing.

7.3 SEQUENCE DIAGRAM

Key sequences include:

- User registration and login

- Room booking process

- Administrator managing bookings

7.4 ACTIVITY DIAGRAM

Main activities include:

- User authentication workflow

- Room booking workflow

- Booking management workflow

SYSTEM TESTING

8. SYSTEM TESTING

8.1 TESTS

The application can be tested using Jest for unit testing and Cypress for end-to end testing.

8.2 TYPE OF TESTING

Various testing methodologies are applicable including unit testing, integration testing, and end-to-end testing.

8.3 UNIT TESTING

Individual components and functions can be tested in isolation.

8.4 INTEGRATION TESTING

Testing interactions between components and modules.

8.5 FUNCTIONAL TESTING

Testing the application's features against functional requirements.

8.6 SYSTEM TESTING

Testing the application as a whole in a production-like environment.

8.7 WHITE BOX TESTING

Testing based on knowledge of the internal structure of the application.

SOURCE CODE

9. SOURCE CODE

Key Components:

1. Backend
   - app.js: Main application logic and API routing

```
  import express from 'express';
import bodyParser from 'body-parser';
import dotenv from 'dotenv';
import cors from 'cors';

import authRoutes from './routes/authRoutes.js';
import roomRoutes from './routes/roomRoutes.js';
import bookingRoutes from
'./routes/bookingRoutes.js'; import adminRoutes from
'./routes/adminRoutes.js';
import errorMiddleware from './middlewares/errorMiddleware.js';
import { sanitizeRequestBody } from './middlewares/sanitizationMiddleware.js';

const app = express();
dotenv.config();

// Configure CORS for both development and production
const allowedOrigins = [
  "http://localhost:5173",
  "https://zorp-hotel.vercel.app",
  "https://zorp-hotel-backend.vercel.app"
];

// Apply CORS middleware with simple configuration
app.use(cors({
  origin: '*',
  methods: ['GET', 'POST', 'PUT', 'DELETE', 'OPTIONS'],
  allowedHeaders: ['Content-Type', 'Authorization']
}));

// Handle OPTIONS requests directly
app.options('*', (req, res) =>
{ res.header('Access-Control-Allow-Origin',
'*');
  res.header('Access-Control-Allow-Methods', 'GET, POST, PUT, DELETE, OPTIONS');
  res.header('Access-Control-Allow-Headers', 'Content-Type, Authorization');
  res.status(200).send();
});

// Middleware
app.use(bodyParser.json());
app.use(bodyParser.urlencoded({ extended: true }));
```

```
app.use(sanitizeRequestBody);

// Set up routes
app.get('/', (req, res) => {
   res.send('API is running...');
});

app.get('/health', (req, res) =>
   { res.status(200).send('API is healthy');
})

// Test endpoint for CORS
app.get('/test-cors', (req, res) => {
   res.status(200).json({   messag
      e: 'CORS is working!',
      origin: req.headers.origin || 'No origin header'
   });
});

app.use('/api/auth', authRoutes);
app.use('/api/rooms', roomRoutes);
app.use('/api/bookings', bookingRoutes);
app.use('/api/admin', adminRoutes);

// Fix for direct routes without /api prefix
app.use('/auth', authRoutes);
app.use('/rooms', roomRoutes);
app.use('/bookings', bookingRoutes);
app.use('/admin', adminRoutes);

app.use(errorMiddleware);

export default app;


  - server.js: Server configuration and startup

import app from './app.js';
import connectDB from './utils/db.js';

connectDB();

const PORT = process.env.PORT || 3000;

app.listen(PORT, () => {
   console.log(`Server is running on port ${PORT}`);
});
```

```
export default app;

  - models: Data models for MongoDB

import mongoose from 'mongoose';

const bookingSchema = new mongoose.Schema({
   user: { type: mongoose.Schema.Types.ObjectId, ref: 'User', required: true },
   room: { type: mongoose.Schema.Types.ObjectId, ref: 'Room', required: true },
   startDate: { type: Date, required: true },
   endDate: { type: Date, required: true },
   status: { type: String, enum: ['booked', 'cancelled'], default: 'booked' }
});

const Booking = mongoose.model('Booking', bookingSchema);

export default Booking;

import mongoose from 'mongoose';

const roomSchema = new
   mongoose.Schema({ type: { type: String,
   required: true },
   price: { type: Number, required: true },
   description: { type: String, default: 'Experience luxury and comfort in our beautifully
designed accommodation.' },
   capacity: { type: Number, default: 2 },
   size: { type: Number, default: 300 }, // in sq ft
   amenities: [String],
   images: {
      type: [String],
      default:   ['https://images.pexels.com/photos/271624/pexels-photo-271624.jpeg']
   },
   availability: { type: Boolean, default: true },
   featured: { type: Boolean, default: false }, // Featured rooms appear on landing page
   createdAt: { type: Date, default: Date.now }
});

// Ensure the model returns placeholder image if no images provided
roomSchema.pre('save', function(next) {
   if (!this.images || this.images.length === 0) {
      this.images  =  ['https://images.pexels.com/photos/271624/pexels-photo-271624.jpeg'];
   }
   next();
});

const Room = mongoose.model('Room', roomSchema);
```

```
export default Room;

import mongoose from 'mongoose';
import bcrypt from 'bcryptjs';

const userSchema = new mongoose.Schema({
   username: { type: String, required: true, unique: true },
   email: { type: String, required: true, unique: true },
   password: { type: String, required: true },
   role: { type: String, enum: ['user', 'admin'], default: 'user' }
});

userSchema.pre('save', async function(next) {
   if (!this.isModified('password')) return next();
   const salt = await bcrypt.genSalt(10);
   this.password = await bcrypt.hash(this.password, salt);
   next();
});

userSchema.methods.matchPassword = async function(enteredPassword)
   { return await bcrypt.compare(enteredPassword, this.password);
};

const User = mongoose.model('User', userSchema);

export default User;

  - controllers: Business logic and data processing

import { register as registerService, login as loginService, changePassword as
changePasswordService } from '../services/authService.js';

export const register = async (req, res, next) =>
   { try {
      const user = await registerService(req.body);
      res.status(201).json(user);
   } catch (error) {
      next(error);
   }
};

export const login = async (req, res, next) =>
   { try {
      const token = await loginService(req.body);
      res.status(200).json({ token });
   } catch (error) {
```

```
        next(error);
    }
};

export const changePassword = async (req, res, next) =>
    { try {
        const result = await changePasswordService(req.user.id, req.body);
        res.status(200).json(result);
    } catch (error) {
        next(error);
    }
};

import Booking from '../models/Booking.js';
import { bookRoom as bookRoomService, getBookingHistory as getBookingHistoryService,
cancelBooking as cancelBookingService } from '../services/bookingService.js';
import Room from '../models/Room.js';
import adminMiddleware from '../middlewares/adminMiddleware.js';
import { validationResult } from 'express-validator';

export const bookRoom = async (req, res, next) =>
    { try {
        const booking = await bookRoomService(req.user, req.body);
        res.status(201).json(booking);
    } catch (error) {
        next(error);
    }
};

export const getBookingHistory = async (req, res, next) => {
    try {
        const bookings = await getBookingHistoryService(req.user);
        res.status(200).json(bookings);
    } catch (error) {
        next(error);
    }
};

export const cancelBooking = async (req, res, next) =>
    { try {
        const booking = await cancelBookingService(req.params.id);
        res.status(200).json(booking);
    } catch (error) {
        next(error);
    }
};
```

```
export const deleteBooking = async (req, res, next) =>
  { try {
      const booking = await Booking.findByIdAndDelete(req.params.id);
      if (!booking) {
        return res.status(404).json({ message: 'Booking not found' });
      }
      res.status(200).json({ message: 'Booking deleted successfully' });
  } catch (error) {
      next(error);
  }
};

export const getAllBookings = async (req, res, next) =>
  { try {
      const bookings = await Booking.find().populate('user room');
      res.status(200).json(bookings);
  } catch (error) {
      next(error);
  }
};

export const updateBooking = async (req, res, next) =>
  { try {
      const booking = await Booking.findByIdAndUpdate(req.params.id, req.body, { new: true
});
      if (!booking) {
        return res.status(404).json({ message: 'Booking not found' });
      }
      res.status(200).json(booking);
  } catch (error) {
      next(error);
  }
};

export const validateRequest = (req, res, next) =>
  { const errors = validationResult(req);
  if (!errors.isEmpty()) {
      return res.status(400).json({ errors: errors.array() });
  }
  next();
};

import Room from '../models/Room.js';
import { listRooms as listRoomsService, searchRooms as searchRoomsService } from
'../services/roomService.js';

export const listRooms = async (req, res, next) => {
```

```
  try {
    const rooms = await listRoomsService();
    res.status(200).json(rooms);
  } catch (error) {
    next(error);
  }
};

export const searchRooms = async (req, res, next) =>
  { try {
    const rooms = await searchRoomsService(req.query);
    res.status(200).json(rooms);
  } catch (error) {
    next(error);
  }
};

export const getFeaturedRooms = async (req, res, next) =>
  { try {
    const rooms = await Room.find({ featured: true }).limit(6);
    res.status(200).json(rooms);
  } catch (error) {
    next(error);
  }
};

export const createRoom = async (req, res, next) =>
  { try {
    // Extract room data from request body
    const { type, price, description, capacity, size, amenities, images, availability, featured }
= req.body;

    // Create room with all possible fields
    const room = new Room({
      type,
      price,
      description,
      capacity,
      size,
      amenities: amenities || [],
      images: images || [],
      availability: availability !== undefined ? availability : true,
      featured: featured !== undefined ? featured : false
    });

    await room.save();
    res.status(201).json(room);
```

```
    } catch (error) {
        next(error);
    }
};

export const updateRoom = async (req, res, next) => {
    try {
        const room = await Room.findByIdAndUpdate(req.params.id, req.body, { new: true });
        if (!room) {
            return res.status(404).json({ message: 'Room not found' });
        }
        res.status(200).json(room);
    } catch (error) {
        next(error);
    }
};

export const deleteRoom = async (req, res, next) =>
    { try {
        const room = await Room.findByIdAndDelete(req.params.id);
        if (!room) {
            return res.status(404).json({ message: 'Room not found' });
        }
        res.status(200).json({ message: 'Room deleted successfully' });
    } catch (error) {
        next(error);
    }
};

import User from '../models/User.js';
import { getUserProfile as getUserProfileService, updateUserProfile as
updateUserProfileService } from '../services/userService.js';

export const getUserProfile = async (req, res, next) =>
    { try {
        const user = await getUserProfileService(req.user);
        res.status(200).json(user);
    } catch (error) {
        next(error);
    }
};

export const updateUserProfile = async (req, res, next) => {
    try {
        const user = await updateUserProfileService(req.user, req.body);
        res.status(200).json(user);
    } catch (error) {
```

```
        next(error);
    }
};

export const getAllUsers = async (req, res, next) =>
    { try {
        const users = await User.find().select('-password');
        res.status(200).json(users);
    } catch (error) {
        next(error);
    }
};

export const updateUser = async (req, res, next) =>
    { try {
        const user = await User.findByIdAndUpdate(req.params.id, req.body, { new: true
}).select('-password');
        if (!user) {
            return res.status(404).json({ message: 'User not found' });
        }
        res.status(200).json(user);
    } catch (error) {
        next(error);
    }
};

export const deleteUser = async (req, res, next) =>
    { try {
        const user = await User.findByIdAndDelete(req.params.id);
        if (!user) {
            return res.status(404).json({ message: 'User not found' });
        }
        res.status(200).json({ message: 'User deleted successfully' });
    } catch (error) {
        next(error);
    }
};
```

- routes: API endpoint definitions

```
  import express from 'express';
import { getAllUsers, updateUser, deleteUser } from '../controllers/userController.js';
import { createRoom, updateRoom, deleteRoom, listRooms } from
'../controllers/roomController.js';
```

```
import { getAllBookings, updateBooking, deleteBooking } from
'../controllers/bookingController.js';
import authMiddleware from '../middlewares/authMiddleware.js';
import adminMiddleware from '../middlewares/adminMiddleware.js';

const router = express.Router();

// Admin routes for user management
router.get('/users', authMiddleware, adminMiddleware, getAllUsers);
router.put('/users/:id', authMiddleware, adminMiddleware, updateUser);
router.delete('/users/:id', authMiddleware, adminMiddleware, deleteUser);

// Admin routes for room management
router.get('/rooms', authMiddleware, adminMiddleware, listRooms);
router.post('/rooms', authMiddleware, adminMiddleware, createRoom);
router.put('/rooms/:id', authMiddleware, adminMiddleware, updateRoom);
router.delete('/rooms/:id', authMiddleware, adminMiddleware, deleteRoom);

// Admin routes for booking management
router.get('/bookings', authMiddleware, adminMiddleware, getAllBookings);
router.put('/bookings/:id', authMiddleware, adminMiddleware, updateBooking);
router.delete('/bookings/:id', authMiddleware, adminMiddleware, deleteBooking);

export default router;

import express from 'express';
import { listRooms, searchRooms, getFeaturedRooms } from
'../controllers/roomController.js';

const router = express.Router();

router.get('/', listRooms);
router.get('/search', searchRooms);
router.get('/featured', getFeaturedRooms);

import express from 'express';
import { bookRoom, getBookingHistory, cancelBooking } from
'../controllers/bookingController.js';
import authMiddleware from '../middlewares/authMiddleware.js';
import { body } from 'express-validator';
import { validateRequest } from '../middlewares/validationMiddleware.js';

const router = express.Router();

router.post('/', [
  body('room').isMongoId().withMessage('Valid room ID is required'),
  body('startDate').isISO8601().withMessage('Valid start date is required'),
```

```
    body('endDate').isISO8601().withMessage('Valid end date is required')
      .custom((endDate, { req }) => new Date(endDate) > new Date(req.body.startDate))
      .withMessage('End date must be after start date'),
    validateRequest
], authMiddleware, bookRoom);

router.get('/history', authMiddleware, getBookingHistory);
router.delete('/:id', authMiddleware, cancelBooking);

export default router;

import express from 'express';
import { register, login, changePassword } from '../controllers/authController.js';
import { getUserProfile } from '../controllers/userController.js';
import authMiddleware from '../middlewares/authMiddleware.js';

const router = express.Router();

// Handle OPTIONS requests explicitly for each route
router.options('/register', (req, res) => {
  res.header('Access-Control-Allow-Origin', req.headers.origin);
  res.header('Access-Control-Allow-Methods', 'POST, OPTIONS');
  res.header('Access-Control-Allow-Headers', 'Content-Type, Authorization');
  res.status(200).send();
});

router.options('/login', (req, res) => {
  res.header('Access-Control-Allow-Origin', req.headers.origin);
  res.header('Access-Control-Allow-Methods', 'POST, OPTIONS');
  res.header('Access-Control-Allow-Headers', 'Content-Type, Authorization');
  res.status(200).send();
});

router.options('/profile', (req, res) => {
  res.header('Access-Control-Allow-Origin', req.headers.origin);
  res.header('Access-Control-Allow-Methods', 'GET, OPTIONS');
  res.header('Access-Control-Allow-Headers', 'Content-Type, Authorization');
  res.status(200).send();
});

router.options('/change-password', (req, res) =>
  { res.header('Access-Control-Allow-Origin',
  req.headers.origin); res.header('Access-Control-Allow-
  Methods', 'POST, OPTIONS');
  res.header('Access-Control-Allow-Headers', 'Content-Type, Authorization');
  res.status(200).send();
});
```

```
router.post('/register', register);
router.post('/login', login);
router.get('/profile', authMiddleware, getUserProfile);
router.post('/change-password', authMiddleware, changePassword);

export default router;
```

- middlewares: Authentication and request processing

```
import User from '../models/User.js';

const adminMiddleware = async (req, res, next) => {
    // Skip admin check for OPTIONS requests (CORS preflight)
    if (req.method === 'OPTIONS') {
        return next();
    }

    if (!req.user || req.user.role !== 'admin') {
        return res.status(403).json({ message: 'Access denied: Admin privileges required' });
    }
    next();
};

export default adminMiddleware;

import jwt from 'jsonwebtoken';
import User from '../models/User.js';

const authMiddleware = async (req, res, next) => {
    // Skip authentication for OPTIONS requests (CORS preflight)
    if (req.method === 'OPTIONS') {
        return next();
    }

    const authHeader = req.header('Authorization');

    if (!authHeader) {
        return res.status(401).json({ message: 'No token, authorization denied' });
    }

    const token = authHeader.startsWith('Bearer ') ? authHeader.substring(7) : authHeader;

    if (!token) {
        return res.status(401).json({ message: 'No token, authorization denied' });
    }

    try {
```

```
      const decoded = jwt.verify(token, process.env.JWT_SECRET);
      req.user = await User.findById(decoded.id).select('-password');

      if (!req.user) {
        return res.status(404).json({ message: 'User not found' });
      }

      next();
    } catch (error) {
      res.status(401).json({ message: 'Token is not valid, authentication error' });
    }
};

export default authMiddleware;

import cors from 'cors';

// Configure CORS options
const corsOptions =
{ origin: [
    "http://localhost:5173",
    "https://zorp-hotel.vercel.app",
    "https://zorp-hotel-backend.vercel.app"
  ],
  credentials: true,
  methods: ['GET', 'POST', 'PUT', 'DELETE', 'OPTIONS'],
  allowedHeaders: ['Content-Type', 'Authorization']
};

// Main CORS middleware
export const corsMiddleware = cors(corsOptions);

// Preflight CORS handler
export const preflightHandler = (req, res, next) => {
  // Handle OPTIONS method
  if (req.method === 'OPTIONS') {
    // Get the origin from the request headers
    const origin = req.headers.origin;

    // Check if the origin is in our allowed origins
    if (corsOptions.origin.includes(origin)) {
      // Set CORS headers
      res.header('Access-Control-Allow-Origin',  origin);
      res.header('Access-Control-Allow-Methods', 'GET, POST, PUT, DELETE,
      OPTIONS'); res.header('Access-Control-Allow-Headers', 'Content-Type,
      Authorization'); res.header('Access-Control-Allow-Credentials', 'true');
```

```
      // Respond with 200 OK
      return res.status(200).send();
    }
  }

  // Pass to next middleware
  next();
};

export default { corsMiddleware, preflightHandler };

const errorMiddleware = (err, req, res, next) => {
    // Log the error for debugging (but not in production response)
    console.error('Error:', err);

    // Handle CORS errors
    if (err.message === 'Not allowed by CORS') {
        return res.status(403).json({
            success: false,
            message: 'CORS error: Origin not allowed'
        });
    }

    // Handle OPTIONS requests with a 200 OK response
    if (req.method === 'OPTIONS') {
        return res.status(200).end();
    }

    // Default error status and message
    const status = err.statusCode || 500;

    // Customize error messages based on environment
    const isProd = process.env.NODE_ENV === 'production';

    // In production, don't send stack traces and internal error details
    const message = isProd && status === 500
        ? 'Internal server error'
        : err.message || 'Something went wrong';

    // Return sanitized error response
    res.status(status).json({
        success: false,
        message,
        // Only include error details in development
        ...(isProd ? {} : { stack: err.stack })
    });
};
```

```
export default errorMiddleware;

export const sanitizeRequestBody = (req, res, next) => {
  // Skip sanitization for OPTIONS requests (CORS preflight)
  if (req.method === 'OPTIONS') {
    return next();
  }

  if (req.body) {
    const sanitized = {};

    // Recursively sanitize all string values
    const sanitizeObject = (obj) => {
      const result = {};

      Object.keys(obj).forEach(key =>
        { const value = obj[key];

        if (typeof value === 'string') {
          // Basic sanitization - remove HTML tags
          result[key] = value.replace(/<[^>]*>?/gm, '');
        } else if (value && typeof value === 'object' && !Array.isArray(value)) {
          // Recursively sanitize nested objects
          result[key] = sanitizeObject(value);
        } else if (Array.isArray(value)) {
          // Sanitize arrays
          result[key] = value.map(item =>
            typeof item === 'string' ? item.replace(/<[^>]*>?/gm, '') :
            (item && typeof item === 'object') ? sanitizeObject(item) : item
          );
        } else {
          // Keep other values as is
          result[key] = value;
        }
      });

      return result;
    };

    req.body = sanitizeObject(req.body);
  }

  next();
};

export const sanitizeRequestBody = (req, res, next) => {
```

```
  // Skip sanitization for OPTIONS requests (CORS preflight)
  if (req.method === 'OPTIONS') {
    return next();
  }

  if (req.body) {
    const sanitized = {};

    // Recursively sanitize all string values
    const sanitizeObject = (obj) => {
      const result = {};

      Object.keys(obj).forEach(key =>
        { const value = obj[key];

        if (typeof value === 'string') {
          // Basic sanitization - remove HTML tags
          result[key] = value.replace(/<[^>]*>?/gm, '');
        } else if (value && typeof value === 'object' && !Array.isArray(value)) {
          // Recursively sanitize nested objects
          result[key] = sanitizeObject(value);
        } else if (Array.isArray(value)) {
          // Sanitize arrays
          result[key] = value.map(item =>
            typeof item === 'string' ? item.replace(/<[^>]*>?/gm, '') :
            (item && typeof item === 'object') ? sanitizeObject(item) : item
          );
        } else {
          // Keep other values as is
          result[key] = value;
        }
      });

      return result;
    };

    req.body = sanitizeObject(req.body);
  }

  next();
};

import { validationResult } from 'express-validator';

export const validateRequest = (req, res, next) => {
    // Skip validation for OPTIONS requests (CORS preflight)
    if (req.method === 'OPTIONS') {
```

```
        return next();
    }

    const errors = validationResult(req);
    if (!errors.isEmpty()) {
        return res.status(400).json({ errors: errors.array() });
    }
    next();
};
```

- utils: Helper functions

```
import mongoose from 'mongoose';

const connectDB = async () => {
    try {
        await mongoose.connect(process.env.DATABASE_URL);
        console.log('MongoDB connected :', mongoose.connection.name);
    } catch (error) {
        console.error('MongoDB connection error:', error);
        process.exit(1);
    }
};

export default connectDB;
```

2. Frontend

- App.jsx: Main application component

```
import { BrowserRouter, Routes, Route, Navigate } from 'react-router-dom'
import { useState, useEffect } from 'react';
import { isAuthenticated, getCurrentUser } from './services/authService';
import Landing from './pages/Landing'
import About from './pages/About'
import Contact from './pages/Contact'
import Booking from './pages/Booking'
import Login from './pages/Login'
import Register from './pages/Register'
import Dashboard from './pages/Dashboard'
import AdminDashboard from './pages/AdminDashboard'
import NotFound from './pages/notFound'

// Protected route component
const ProtectedRoute = ({ children }) =>
  { const isAuth = isAuthenticated();
  return isAuth ? children : <Navigate to="/login" />;
};
```

```
// Admin route component
const AdminRoute = ({ children }) =>
  { const isAuth = isAuthenticated();
  if (!isAuth) {
    return <Navigate to="/login" />;
  }

  const user = getCurrentUser();
  if (user?.role !== 'admin') {
    return <Navigate to="/dashboard" />;
  }

  return children;
};

const App = () => {
  const [checking, setChecking] = useState(true);

  useEffect(() => {
    // Simulate checking auth status
    setChecking(false);
  }, []);

  if (checking) {
    return <div className="h-screen flex items-center justify-center">
      <div className="animate-spin rounded-full h-12 w-12 border-b-2 border-indigo-
600"></div>
    </div>;
  }

  return (
    <BrowserRouter>
      <Routes>
        <Route path="/" element={<Landing />} />
        <Route path="/about" element={<About />} />
        <Route path="/contact" element={<Contact />} />
        <Route path="/booking" element={<Booking />} />
        <Route path="/login" element={<Login />} />
        <Route path="/register" element={<Register />} />
        <Route
          path="/dashboard"
          element={
            <ProtectedRoute>
              <Dashboard />
            </ProtectedRoute>
          }
```

```
      />
      <Route
        path="/admin"
        element={
          <AdminRoute>
            <AdminDashboard />
          </AdminRoute>
        }
      />
      <Route path="/*" element={<NotFound />} />
    </Routes>
  </BrowserRouter>
 );
}

export default App;
```

- pages: Individual pages for different routes

```
    import Nav from "../components/Nav";
import { Link } from "react-router-dom";

const About = () =>
 { return (
  <>
    <Nav />

    {/* Hero Section */}
    <section className="pt-32 pb-20 bg-indigo-50">
      <div className="container mx-auto px-4">
        <div className="max-w-3xl mx-auto text-center">
          <h1 className="text-4xl md:text-5xl font-serif font-bold text-gray-800 mb-6">About
Zorp Hotel</h1>
          <p className="text-xl text-gray-600 mb-8">Discover the story behind our luxury
hotel and our commitment to exceptional service.</p>
        </div>
      </div>
    </section>

    {/* History Section */}
    <section className="py-16">
      <div className="container mx-auto px-4">
        <div className="flex flex-col md:flex-row items-center gap-12">
          <div className="md:w-1/2">
            <img src="https://images.pexels.com/photos/1134176/pexels-photo-1134176.jpeg"
alt="Hotel History" className="rounded-lg shadow-lg w-full" />
          </div>
```

```
          <div className="md:w-1/2">
            <h2 className="text-3xl font-serif font-bold text-gray-800 mb-6">Our History</h2>
            <p className="text-gray-600 mb-4">Founded in 2005, Zorp Hotel began as a small boutique establishment with a clear vision: to provide an unparalleled hospitality experience that combines luxury with authentic personalized service.</p>
            <p className="text-gray-600 mb-4">Over the years, we've grown to become one of the most prestigious hotels in the region, known for our attention to detail and commitment to guest satisfaction.</p>
            <p className="text-gray-600">Today, Zorp Hotel stands as a landmark of luxury and excellence, continually evolving to meet the changing needs of our discerning guests while maintaining our core values of exceptional service and attention to detail.</p>
          </div>
        </div>
      </div>
    </section>

    {/* Mission & Vision Section */}
    <section className="py-16 bg-gray-50">
      <div className="container mx-auto px-4">
        <div className="max-w-4xl mx-auto">
          <div className="text-center mb-12">
            <h2 className="text-3xl font-serif font-bold text-gray-800 mb-4">Our Mission & Vision</h2>
            <p className="text-gray-600">Guiding principles that define our approach to hospitality</p>
          </div>

          <div className="grid md:grid-cols-2 gap-8">
            <div className="bg-white p-8 rounded-lg shadow-md">
              <div className="w-16 h-16 bg-indigo-100 rounded-full flex items-center justify-center mb-6 mx-auto">
                <svg className="w-8 h-8 text-indigo-600" fill="none" stroke="currentColor" viewBox="0 0 24 24" xmlns="http://www.w3.org/2000/svg">
                  <path strokeLinecap="round" strokeLinejoin="round" strokeWidth={2} d="M13 10V3L4 14h7v7l9-11h-7z" />
                </svg>
              </div>
              <h3 className="text-xl font-bold text-center mb-4 text-gray-800">Our Mission</h3>
              <p className="text-gray-600 text-center">To create memorable experiences for our guests through impeccable service, luxurious accommodations, and attention to every detail.</p>
            </div>

            <div className="bg-white p-8 rounded-lg shadow-md">
              <div className="w-16 h-16 bg-indigo-100 rounded-full flex items-center justify-center mb-6 mx-auto">
```

```
              <svg className="w-8 h-8 text-indigo-600" fill="none" stroke="currentColor"
viewBox="0 0 24 24" xmlns="http://www.w3.org/2000/svg">
                <path strokeLinecap="round" strokeLinejoin="round" strokeWidth={2} d="M15
12a3 3 0 11-6 0 3 3 0 016 0z" />
                <path strokeLinecap="round" strokeLinejoin="round" strokeWidth={2}
d="M2.458 12C3.732 7.943 7.523 5 12 5c4.478 0 8.268 2.943 9.542 7-1.274 4.057-5.064 7-
9.542 7-4.477 0-8.268-2.943-9.542-7z" />
              </svg>
            </div>
            <h3 className="text-xl font-bold text-center mb-4 text-gray-800">Our Vision</h3>
            <p className="text-gray-600 text-center">To be recognized as the leading luxury
hotel, setting the standard for excellence in hospitality and creating a legacy of
unforgettable guest experiences.</p>
          </div>
        </div>
      </div>
    </section>

    {/* Team Section */}
    <section className="py-16">
      <div className="container mx-auto px-4">
        <div className="text-center mb-12">
          <h2 className="text-3xl font-serif font-bold text-gray-800 mb-4">Our Leadership
Team</h2>
          <p className="text-gray-600 max-w-2xl mx-auto">Meet the talented professionals
who work tirelessly to ensure your stay exceeds expectations.</p>
        </div>

        <div className="grid md:grid-cols-3 gap-8 max-w-5xl mx-auto">
          <div className="bg-white rounded-lg overflow-hidden shadow-md hover:shadow-lg
transition-shadow">
            <img src="https://images.pexels.com/photos/2182970/pexels-photo-2182970.jpeg"
alt="CEO" className="w-full h-64 object-cover object-top" />
            <div className="p-6">
              <h3 className="text-xl font-bold mb-1 text-gray-800">Shah Rukh Khan</h3>
              <p className="text-indigo-600 mb-4">Chief Executive Officer</p>
              <p className="text-gray-600">With his charismatic leadership and visionary
approach, Shah Rukh has transformed Zorp Hotel into a landmark of luxury hospitality.</p>
            </div>
          </div>

          <div className="bg-white rounded-lg overflow-hidden shadow-md hover:shadow-lg
transition-shadow">
            <img src="https://images.pexels.com/photos/3756679/pexels-photo-3756679.jpeg"
alt="Operations Director" className="w-full h-64 object-cover object-top" />
            <div className="p-6">
```

```
            <h3 className="text-xl font-bold mb-1 text-gray-800">Priyanka Chopra</h3>
            <p className="text-indigo-600 mb-4">Director of Operations</p>
            <p className="text-gray-600">Priyanka's international expertise and attention to detail ensures that every aspect of our hotel runs with precision and excellence.</p>
          </div>
        </div>

        <div className="bg-white rounded-lg overflow-hidden shadow-md hover:shadow-lg transition-shadow">
          <img src="https://images.pexels.com/photos/3814446/pexels-photo-3814446.jpeg" alt="Culinary Director" className="w-full h-64 object-cover object-top" />
          <div className="p-6">
            <h3 className="text-xl font-bold mb-1 text-gray-800">Sanjeev Kapoor</h3>
            <p className="text-indigo-600 mb-4">Culinary Director</p>
            <p className="text-gray-600">India's most celebrated chef brings his culinary mastery to create unforgettable dining experiences that blend tradition with innovation.</p>
          </div>
        </div>
      </div>
    </div>
  </section>

  {/* CTA Section */}
  <section className="py-16 bg-indigo-600">
    <div className="container mx-auto px-4">
      <div className="max-w-3xl mx-auto text-center">
        <h2 className="text-3xl font-serif font-bold text-white mb-6">Experience Zorp Hotel Yourself</h2>
        <p className="text-xl text-indigo-100 mb-8">Book your stay today and discover why our guests keep returning year after year.</p>
        <Link to="/booking" className="inline-block bg-white text-indigo-600 px-8 py-4 rounded-lg font-bold hover:bg-gray-100 transition-colors">Book Your Stay</Link>
      </div>
    </div>
  </section>

  {/* Footer included via Nav component */}
 </>
 );
}

export default About;

import { useState, useEffect } from 'react';
import { Navigate } from 'react-router-dom';
import Nav from "../components/Nav";
```

```
import { getCurrentUser } from '../services/authService';
import { getAllUsers, deleteUser, updateUser } from '../services/adminService';
import { getAllRooms, createRoom, updateRoom, deleteRoom } from
'../services/adminService';
import { getAllBookings, updateBooking } from '../services/adminService';
import { toast, ToastContainer } from 'react-toastify';
import 'react-toastify/dist/ReactToastify.css';

const RoomsManagement = ({ rooms, onRefresh }) => {
  const [editingRoom, setEditingRoom] = useState(null);
  const [roomForm, setRoomForm] = useState({
    _id: '',
    type: '',
    price: '',
    description: '',
    capacity: 2,
    size: 300,
    amenities: '',
    images: '',
    availability: true,
    featured: false
  });

  const [newRoomForm, setNewRoomForm] = useState({
    type: '',
    price: '',
    description: '',
    availability: true
  });

  const handleEditRoom = (room) => {
    setEditingRoom(room._id);
    setRoomForm({
      _id: room._id,
      type: room.type,
      price: room.price,
      description: room.description || '',
      capacity: room.capacity || 2,
      size: room.size || 300,
      amenities: room.amenities?.join(', ') || '',
      images: room.images?.join(', ') || '',
      availability: room.availability !== undefined ? room.availability : true,
      featured: room.featured !== undefined ? room.featured : false
    });
  };

  const handleCancelEdit = () => {
```

```
  setEditingRoom(null);
  setRoomForm({
    _id: '',
    type: '',
    price: '',
    description: '',
    capacity: 2,
    size: 300,
    amenities: '',
    images: '',
    availability: true,
    featured: false
  });
};

const handleRoomUpdate = async (roomId) =>
  { try {
    const formattedRoom = {
      ...roomForm,
      amenities: roomForm.amenities.split(',').map(item => item.trim()).filter(Boolean),
      images: roomForm.images.split(',').map(item => item.trim()).filter(Boolean)
    };
    await updateRoom(roomId, formattedRoom);
    toast.success('Room updated successfully');
    setEditingRoom(null);
    onRefresh();
  } catch (error) {
    toast.error(error.message || 'Failed to update room');
  }
};

const handleDeleteRoom = async (roomId) => {
  if (window.confirm('Are you sure you want to delete this room?'))
    { try {
      await deleteRoom(roomId);
      toast.success('Room deleted successfully');
      onRefresh();
    } catch (error) {
      toast.error(error.message || 'Failed to delete room');
    }
  }
};

const handleInputChange = (e) => {
  const { name, value, type, checked } = e.target;
  setRoomForm(prev => ({
    ...prev,
```

```
    [name]: type === 'checkbox' ? checked : name === 'price' ? Number(value) : value
  }));
};

const handleNewRoomInputChange = (e) =>
  { const { name, value, type, checked } = e.target;
  setNewRoomForm(prev => ({
    ...prev,
    [name]: type === 'checkbox' ? checked : name === 'price' ? Number(value) : value
  }));
};

const handleAddRoom = async () => {
  // Validate form
  if (!newRoomForm.type || !newRoomForm.price)
    { toast.error('Room type and price are required');
    return;
  }

  try {
    await createRoom(newRoomForm);
    toast.success('Room added successfully');
    // Reset form
    setNewRoomForm({ ty
    pe: '',
      price: '',
      description: '',
      availability: true
    });
    onRefresh();
  } catch (error) {
    toast.error(error.message || 'Failed to add room');
  }
};

return (
  <div>
    <h3 className="text-xl font-bold mb-4">Manage Rooms</h3>

    <div className="bg-indigo-50 p-4 rounded-lg mb-6">
      <h4 className="font-bold mb-2">Add New Room</h4>
      <div className="grid grid-cols-1 md:grid-cols-4 gap-4">
        <input
          type="text"
          name="type"
          placeholder="Room Type"
          value={newRoomForm.type}
```

```
          onChange={handleNewRoomInputChange}
          className="px-3 py-2 border rounded"
        />
        <input
          type="number"
          name="price"
          placeholder="Price per Night"
          value={newRoomForm.price}
          onChange={handleNewRoomInputChange}
          className="px-3 py-2 border rounded"
        />
        <input
          type="text"
          name="description"
          placeholder="Description"
          value={newRoomForm.description}
          onChange={handleNewRoomInputChange}
          className="px-3 py-2 border rounded"
        />
        <button
          onClick={handleAddRoom}
          className="bg-indigo-600 text-white px-4 py-2 rounded hover:bg-indigo-700
transition-colors"
        >
          Add Room
        </button>
      </div>
    </div>

    <div className="overflow-x-auto">
      <table className="min-w-full bg-white">
        <thead>
          <tr className="bg-gray-100">
            <th className="py-2 px-4 text-left">Room Type</th>
            <th className="py-2 px-4 text-left">Price</th>
            <th className="py-2 px-4 text-left">Description</th>
            <th className="py-2 px-4 text-left">Availability</th>
            <th className="py-2 px-4 text-left">Actions</th>
          </tr>
        </thead>
        <tbody>
          {rooms.map(room => (
            <tr key={room._id} className="border-b">
              <td className="py-2 px-4">
                {editingRoom === room._id ? (
                  <input
                    type="text"
```

```
      name="type"
      value={roomForm.type}
      onChange={handleInputChange}
      className="px-2 py-1 border rounded w-full"
    />
  ) : (
    room.type
  )}
</td>
<td className="py-2 px-4">
  {editingRoom === room._id ? (
    <input
      type="number"
      name="price"
      value={roomForm.price}
      onChange={handleInputChange}
      className="px-2 py-1 border rounded w-full"
    />
  ) : (
    `Rs.${room.price}`
  )}
</td>
<td className="py-2 px-4">
  {editingRoom === room._id ? (
    <input
      type="text"
      name="description"
      value={roomForm.description}
      onChange={handleInputChange}
      className="px-2 py-1 border rounded w-full"
    />
  ) : (
    room.description || '-'
  )}
</td>
<td className="py-2 px-4">
  {editingRoom === room._id ? (
    <select
      name="availability"
      value={roomForm.availability.toString()}
      onChange={handleInputChange}
      className="px-2 py-1 border rounded"
    >
      <option value="true">Available</option>
      <option value="false">Not Available</option>
    </select>
  ) : (
```

```
                room.availability ? 'Available' : 'Not Available'
              )}
            </td>
            <td className="py-2 px-4">
             {editingRoom === room._id ? (
              <div className="flex space-x-2">
               <button
                onClick={() => handleRoomUpdate(room._id)}
                className="px-3 py-1 bg-green-600 text-white rounded text-sm"
               >
                Save
               </button>
               <button
                onClick={handleCancelEdit}
                className="px-3 py-1 bg-gray-600 text-white rounded text-sm"
               >
                Cancel
               </button>
              </div>
             ) : (
              <div className="flex space-x-2">
               <button
                onClick={() => handleEditRoom(room)}
                className="px-3 py-1 bg-indigo-600 text-white rounded text-sm"
               >
                Edit
               </button>
               <button
                onClick={() => handleDeleteRoom(room._id)}
                className="px-3 py-1 bg-red-600 text-white rounded text-sm"
               >
                Delete
               </button>
              </div>
             )}
            </td>
           </tr>
          ))}
         </tbody>
        </table>
       </div>
      </div>
     );
    };

    const BookingsManagement = ({ bookings, onRefresh }) =>
     { const formatDate = (dateString) => {
```

```
  const options = { year: 'numeric', month: 'long', day: 'numeric' };
  return new Date(dateString).toLocaleDateString(undefined, options);
 };

 const handleUpdateStatus = async (bookingId, status) => {
  try {
   await updateBooking(bookingId, { status });
   toast.success(`Booking ${status}`);
   onRefresh();
  } catch (error) {
   toast.error(error.message || 'Failed to update booking');
  }
 };

 return (
  <div>
   <h3 className="text-xl font-bold mb-4">Manage Bookings</h3>
   <div className="overflow-x-auto">
    <table className="min-w-full bg-white">
     <thead>
      <tr className="bg-gray-100">
       <th className="py-2 px-4 text-left">Guest</th>
       <th className="py-2 px-4 text-left">Room</th>
       <th className="py-2 px-4 text-left">Dates</th>
       <th className="py-2 px-4 text-left">Status</th>
       <th className="py-2 px-4 text-left">Actions</th>
      </tr>
     </thead>
     <tbody>
      {bookings.map(booking => (
       <tr key={booking._id} className="border-b">
        <td className="py-2 px-4">{booking.user.username}</td>
        <td className="py-2 px-4">{booking.room.type}</td>
        <td className="py-2 px-4">{formatDate(booking.startDate)} -
{formatDate(booking.endDate)}</td>
        <td className="py-2 px-4">
         <span className={`px-2 py-1 rounded text-xs ${
          booking.status === 'booked' ? 'bg-green-100 text-green-800' : 'bg-red-100 text-
red-800'
         }`}>
          {booking.status.toUpperCase()}
         </span>
        </td>
        <td className="py-2 px-4">
         {booking.status === 'booked' ? (
          <button
           onClick={() => handleUpdateStatus(booking._id, 'cancelled')}
```

```
                  className="px-3 py-1 bg-red-600 text-white rounded text-sm"
                >
                  Cancel
                </button>
              ) : (
                <button
                  onClick={() => handleUpdateStatus(booking._id, 'booked')}
                  className="px-3 py-1 bg-green-600 text-white rounded text-sm"
                >
                  Restore
                </button>
              )}
            </td>
          </tr>
        ))}
      </tbody>
    </table>
  </div>
 </div>
);
};

const UsersManagement = ({ users, onRefresh }) =>
 { const handleDeleteUser = async (userId) => {
  if (window.confirm('Are you sure you want to delete this user?'))
   { try {
    await deleteUser(userId);
    toast.success('User deleted successfully');
    onRefresh();
   } catch (error) {
    toast.error(error.message || 'Failed to delete user');
   }
  }
 };

 const handleToggleRole = async (user) => {
  const newRole = user.role === 'admin' ? 'user' : 'admin';
  try {
   await updateUser(user._id, { role: newRole });
   toast.success(`User role updated to ${newRole}`);
   onRefresh();
  } catch (error) {
   toast.error(error.message || 'Failed to update user role');
  }
 };

 return (
```

```
  <div>
    <h3 className="text-xl font-bold mb-4">Manage Users</h3>
    <div className="overflow-x-auto">
      <table className="min-w-full bg-white">
        <thead>
          <tr className="bg-gray-100">
            <th className="py-2 px-4 text-left">Username</th>
            <th className="py-2 px-4 text-left">Email</th>
            <th className="py-2 px-4 text-left">Role</th>
            <th className="py-2 px-4 text-left">Actions</th>
          </tr>
        </thead>
        <tbody>
          {users.map(user => (
            <tr key={user._id} className="border-b">
              <td className="py-2 px-4">{user.username}</td>
              <td className="py-2 px-4">{user.email}</td>
              <td className="py-2 px-4">
                <span className={`px-2 py-1 rounded text-xs ${
                  user.role === 'admin' ? 'bg-purple-100 text-purple-800' : 'bg-blue-100 text-blue-
800'
                }`}>
                  {user.role.toUpperCase()}
                </span>
              </td>
              <td className="py-2 px-4">
                <div className="flex space-x-2">
                  <button
                    onClick={() => handleToggleRole(user)}
                    className="px-3 py-1 bg-indigo-600 text-white rounded text-sm"
                  >
                    {user.role === 'admin' ? 'Make User' : 'Make Admin'}
                  </button>
                  <button
                    onClick={() => handleDeleteUser(user._id)}
                    className="px-3 py-1 bg-red-600 text-white rounded text-sm"
                  >
                    Delete
                  </button>
                </div>
              </td>
            </tr>
          ))}
        </tbody>
      </table>
    </div>
  </div>
```

```
  );
};

const AdminDashboard = () => {
  const [activeTab, setActiveTab] = useState('rooms');
  const [rooms, setRooms] = useState([]);
  const [users, setUsers] = useState([]);
  const [bookings, setBookings] = useState([]);
  const [loading, setLoading] = useState(true);
  const [user, setUser] = useState(null);

  useEffect(() => {
    const currentUser = getCurrentUser();
    setUser(currentUser);

    fetchData();
  }, [activeTab]);

  const fetchData = async () => {
    setLoading(true);
    try {
      if (activeTab === 'rooms') {
        const roomsData = await getAllRooms();
        setRooms(roomsData);
      } else if (activeTab === 'users') {
        const usersData = await getAllUsers();
        setUsers(usersData);
      } else if (activeTab === 'bookings') {
        const bookingsData = await getAllBookings();
        setBookings(bookingsData);
      }
    } catch (error) {
      console.error(`Error fetching ${activeTab}:`, error);
      toast.error(`Failed to load ${activeTab}. ${error.message}`);
    } finally {
      setLoading(false);
    }
  };

  // Redirect if not admin
  if (user && user.role !== 'admin') {
    return <Navigate to="/dashboard" replace />;
  }

  return (
    <>
      <Nav />
```

```
    <ToastContainer position="top-right" autoClose={5000} />

    {/* Hero Section */}
    <section className="pt-32 pb-20 bg-indigo-50">
      <div className="container mx-auto px-4">
        <h1 className="text-4xl font-serif font-bold text-gray-800 mb-6">Admin
Dashboard</h1>

        {/* Admin tabs */}
        <div className="bg-white shadow-md rounded-lg overflow-hidden mb-8">
          <div className="flex border-b">
            <button
              className={`px-6 py-3 text-lg font-medium ${activeTab === 'rooms' ? 'border-b-2
border-indigo-600 text-indigo-600' : 'text-gray-500 hover:text-gray-700'}`}
              onClick={() => setActiveTab('rooms')}
            >
              Rooms
            </button>
            <button
              className={`px-6 py-3 text-lg font-medium ${activeTab === 'bookings' ? 'border-b-
2 border-indigo-600 text-indigo-600' : 'text-gray-500 hover:text-gray-700'}`}
              onClick={() => setActiveTab('bookings')}
            >
              Bookings
            </button>
            <button
              className={`px-6 py-3 text-lg font-medium ${activeTab === 'users' ? 'border-b-2
border-indigo-600 text-indigo-600' : 'text-gray-500 hover:text-gray-700'}`}
              onClick={() => setActiveTab('users')}
            >
              Users
            </button>
          </div>
        </div>
      </div>
    </section>

    {/* Tab content */}
    <section className="py-8 mb-16">
      <div className="container mx-auto px-4">
        {loading ? (
          <div className="flex justify-center py-12">
            <div className="animate-spin rounded-full h-12 w-12 border-b-2 border-indigo-
600"></div>
          </div>
        ) : (
          <div className="bg-white p-6 rounded-lg shadow-md">
```

```
          {activeTab === 'rooms' && <RoomsManagement rooms={rooms}
onRefresh={fetchData} />}
          {activeTab === 'bookings' && <BookingsManagement bookings={bookings}
onRefresh={fetchData} />}
          {activeTab === 'users' && <UsersManagement users={users} onRefresh={fetchData}
/>}
         </div>
        )}
       </div>
     </section>
   </>
  );
};

export default AdminDashboard;

import { useState, useEffect } from 'react';
import { useNavigate } from 'react-router-dom';
import Nav from "../components/Nav";
import { createBooking } from '../services/bookingService';
import { searchRooms } from '../services/roomService';
import { isAuthenticated } from '../services/authService';
import { toast, ToastContainer } from 'react-toastify';
import 'react-toastify/dist/ReactToastify.css';

const Booking = () => {
  const navigate = useNavigate();
  const [loading, setLoading] = useState(false);
  const [checkingAvailability, setCheckingAvailability] = useState(false);
  const [rooms, setRooms] = useState([]);
  const [currentStep, setCurrentStep] = useState(1);
  const [bookingConfirmation, setBookingConfirmation] = useState(null);

  const [formData, setFormData] =
    useState({ startDate: '',
    endDate: '',
    guests: 1,
    roomId: '',
    firstName: '',
    lastName: '',
    email: '',
    phone: '',
    specialRequests: ''
  });

  useEffect(() => {
    // Fetch available rooms from backend
```

```
  const fetchRooms = async () =>
    { try {
      setLoading(true);
      const availableRooms = await searchRooms({ availability: true });

      if (availableRooms.length > 0) {
        setRooms(availableRooms);
        setFormData(prev => ({
          ...prev,
          roomId: availableRooms[0]._id
        }));
      } else {
        toast.info('No rooms are currently available. Please try different dates.');
      }
    } catch (error) {
      console.error('Error fetching rooms:', error);
      toast.error('Failed to load available rooms. Please try again later.');
    } finally {
      setLoading(false);
    }
  };

  fetchRooms();

  // Check for any pending booking in session storage (from redirects)
  const pendingBooking = sessionStorage.getItem('pendingBooking');
  if (pendingBooking) {
    try {
      const bookingData = JSON.parse(pendingBooking);
      setFormData(prev => ({
        ...prev,
        ...bookingData
      }));
      // Clear session storage
      sessionStorage.removeItem('pendingBooking');
    } catch (error) {
      console.error('Error parsing pending booking:', error);
    }
  }
}, []);

// Effect for date changes - check availability
useEffect(() => {
  if (formData.startDate && formData.endDate && formData.roomId) {
    checkRoomAvailability();
  }
}, [formData.startDate, formData.endDate, formData.roomId]);
```

```
  const checkRoomAvailability = async () => {
    if (!formData.startDate || !formData.endDate || !formData.roomId) return;

    // Validate dates
    const startDate = new Date(formData.startDate);
    const endDate = new Date(formData.endDate);

    if (endDate <= startDate) {
      toast.error('Check-out date must be after check-in date');
      return;
    }

    setCheckingAvailability(true);

    try {
      // Check if selected dates are available for the selected room
      const queryParams = {
        startDate: formData.startDate,
        endDate: formData.endDate,
        roomId: formData.roomId
      };

      const availableRooms = await searchRooms(queryParams);

      // If the selected room is not in the available rooms list
      const isAvailable = availableRooms.some(room => room._id === formData.roomId);

      if (!isAvailable) {
        toast.error('The selected room is not available for these dates. Please choose different
dates or another room.');

        // Reset the room selection or dates
        setFormData(prev => ({
          ...prev,
          roomId: availableRooms.length > 0 ? availableRooms[0]._id : ''
        }));

        // Update available rooms
        setRooms(availableRooms);
      }
    } catch (error) {
      console.error('Error checking availability:', error);
      toast.error('Failed to check room availability. Please try again.');
    } finally {
      setCheckingAvailability(false);
    }
```

```
};

const handleChange = (e) =>
 { const { name, value } =
 e.target; setFormData(prev =>
 ({
   ...prev,
   [name]: value
 }));
};

const calculateStayDuration = () => {
 if (!formData.startDate || !formData.endDate) return 0;

 const start = new Date(formData.startDate);
 const end = new Date(formData.endDate);
 const difference = end.getTime() - start.getTime();
 const days = Math.ceil(difference / (1000 * 3600 * 24));

 return days > 0 ? days : 0;
};

const calculateTotalPrice = () => {
 const selectedRoom = rooms.find(room => room._id === formData.roomId);
 if (!selectedRoom) return { roomCharge: 0, taxes: 0, total: 0 };

 const duration = calculateStayDuration();
 const roomCharge = selectedRoom.price * duration;
 const taxes = Math.round(roomCharge * 0.15);

 return { roomCharge, taxes, total: roomCharge + taxes };
};

const nextStep = () =>
 { if (currentStep === 1)
 {
   // Validate first step
   if (!formData.startDate || !formData.endDate || !formData.roomId)
    { toast.error('Please fill in all required fields');
    return;
   }

   if (calculateStayDuration() <= 0) {
    toast.error('Check-out date must be after check-in date');
    return;
   }

   // Check availability again before proceeding
   checkRoomAvailability().then(() => {
```

```
      setCurrentStep(prev => prev + 1);
    });
  } else {
    setCurrentStep(prev => prev + 1);
  }
};

const prevStep = () =>
  { setCurrentStep(prev => prev - 1);
};

const handleSubmit = async (e) => {
  e.preventDefault();

  // Validate required fields
  if (!formData.firstName || !formData.lastName || !formData.email || !formData.phone) {
    toast.error('Please fill in all required fields');
    return;
  }

  if (!isAuthenticated()) {
    toast.info('Please log in to complete your booking');
    // Store booking details in session storage and redirect to login
    sessionStorage.setItem('pendingBooking', JSON.stringify(formData));
    navigate('/login', { state: { from: '/booking' } });
    return;
  }

  setLoading(true);

  try {
    // Format data for API
    const bookingData =
    { room: formData.roomId,
      startDate: formData.startDate,
      endDate: formData.endDate,
      guestInfo: {
        firstName: formData.firstName,
        lastName: formData.lastName,
        email: formData.email,
        phone: formData.phone,
        specialRequests: formData.specialRequests
      },
      guests: formData.guests
    };

    const response = await createBooking(bookingData);
```

```
      toast.success('Booking successful!');

      // Store booking confirmation
      setBookingConfirmation(response);

      // Clear form and move to confirmation step
      setCurrentStep(3);
    } catch (error) {
      console.error('Booking error:',
      error);

      // Handle specific error cases
      if (error.message && error.message.includes('already booked')) {
        toast.error('This room is already booked for the selected dates. Please choose different
dates or another room.');
      } else if (error.message && error.message.includes('not available')) {
        toast.error('The selected room is no longer available. Please choose another room.');
      } else {
        toast.error(error.message || 'Failed to complete booking. Please try again.');
      }
    } finally {
      setLoading(false);
    }
  };

  const selectedRoom = rooms.find(room => room._id === formData.roomId) || {};
  const { roomCharge, taxes, total } = calculateTotalPrice();
  const stayDuration = calculateStayDuration();

  // Add this function to handle image errors
  const handleImageError = (e) => {
    e.target.src  =  "https://images.pexels.com/photos/271624/pexels-photo-271624.jpeg";
  };

  // Define steps
  const renderStepContent = () =>
    { switch (currentStep) {
      case 1:
        return (
          <>
            {/* Step 1: Select Dates and Room */}
            <div className="bg-white p-6 rounded-lg shadow-md mb-6">
              <h3 className="text-xl font-bold mb-6 text-gray-800 pb-2 border-b">Select
Dates</h3>
              <div className="grid md:grid-cols-2 gap-6">
                <div>
```

```
            <label className="block text-gray-700 mb-2">Check-in Date <span
className="text-red-600">*</span></label>
            <input
              type="date"
              name="startDate"
              value={formData.startDate}
              onChange={handleChange}
              min={new Date().toISOString().split('T')[0]}
              required
              className="w-full px-4 py-3 border border-gray-300 rounded-lg focus:ring
focus:ring-indigo-200 focus:border-indigo-600 outline-none transition"
            />
           </div>
           <div>
            <label className="block text-gray-700 mb-2">Check-out Date <span
className="text-red-600">*</span></label>
            <input
              type="date"
              name="endDate"
              value={formData.endDate}
              onChange={handleChange}
              min={formData.startDate || new Date().toISOString().split('T')[0]}
              required
              className="w-full px-4 py-3 border border-gray-300 rounded-lg focus:ring
focus:ring-indigo-200 focus:border-indigo-600 outline-none transition"
            />
           </div>
          </div>
          <div className="mt-6">
           <label className="block text-gray-700 mb-2">Number of Guests</label>
           <select
            name="guests"
            value={formData.guests}
            onChange={handleChange}
            className="w-full px-4 py-3 border border-gray-300 rounded-lg focus:ring
focus:ring-indigo-200 focus:border-indigo-600 outline-none transition"
           >
            {[1, 2, 3, 4].map(num => (
             <option key={num} value={num}>{num} {num === 1 ? 'Guest' : 'Guests'}</option>
            ))}
           </select>
          </div>

          {checkingAvailability && (
           <div className="mt-4 flex items-center text-indigo-600">
            <svg className="animate-spin -ml-1 mr-3 h-5 w-5"
xmlns="http://www.w3.org/2000/svg" fill="none" viewBox="0 0 24 24">
```

```
              <circle className="opacity-25" cx="12" cy="12" r="10" stroke="currentColor"
strokeWidth="4"></circle>
              <path className="opacity-75" fill="currentColor" d="M4 12a8 8 0 018-
8V0C5.373 0 0 5.373 0 12h4zm2 5.291A7.962 7.962 0 014 12H0c0 3.042 1.135 5.824 3
7.938l3-2.647z"></path>
            </svg>
            Checking room availability...
          </div>
        )}
      </div>

      {/* Room Selection */}
      <div className="bg-white p-6 rounded-lg shadow-md mb-6">
        <h3 className="text-xl font-bold mb-6 text-gray-800 pb-2 border-b">Select Room
Type</h3>

        {loading ? (
          <div className="flex justify-center py-8">
            <div className="animate-spin rounded-full h-12 w-12 border-b-2 border-indigo-
600"></div>
          </div>
        ) : (
          <div className="space-y-4">
            {rooms.length > 0 ?
              ( rooms.map(room => (
                <div
                  key={room._id}
                  className={`border p-4 rounded-lg cursor-pointer transition
${formData.roomId === room._id ? 'border-indigo-600 ring-2 ring-indigo-200' : 'border-gray-
200 hover:border-indigo-300'}`}
                  onClick={() => setFormData(prev => ({ ...prev, roomId: room._id }))}
                >
                  <div className="flex flex-col md:flex-row gap-4">
                    <div className="md:w-1/4">
                      <div className="aspect-video bg-gray-200 rounded-lg overflow-hidden">
                        <img src={room.images?.
                          [0] ||
"https://images.pexels.com/photos/271624/pexels-photo-271624.jpeg"}
                          alt={room.type}
                          className="w-full h-full object-cover"
                          onError={handleImageError}
                        />
                      </div>
                    </div>
                    <div className="md:w-3/4">
                      <div className="flex items-center">
                        <input
```

```
                    type="radio"
                    name="roomId"
                    id={room._id}
                    value={room._id}
                    checked={formData.roomId === room._id}
                    onChange={handleChange}
                    className="w-4 h-4 text-indigo-600 focus:ring-indigo-500"
                   />
                   <label htmlFor={room._id} className="ml-2 flex-1 cursor-pointer">
                    <span className="block font-bold text-lg text-gray-
800">{room.type}</span>
                    <p className="text-gray-600 mb-2">{room.description}</p>
                    <span className="block text-indigo-600 text-xl font- bold">Rs.
{room.price} <span className='text-base text-indigo-400'>/night</span></span>
                   </label>
                  </div>
                 </div>
                </div>
               ))
              ) : (
               <div className="text-center py-8">
                <p className="text-gray-600 mb-4">No rooms available for the selected
dates.</p>
                <p className="text-gray-600">Please try different dates or contact us
directly.</p>
               </div>
              )}
             </div>
            )}
           </div>

           <div className="flex justify-end">
            <button
             type="button"
             onClick={nextStep}
             disabled={loading || checkingAvailability || rooms.length === 0}
             className="bg-indigo-600 text-white py-3 px-8 rounded-lg font-medium hover:bg-
indigo-700 transition-colors disabled:bg-indigo-400 disabled:cursor-not-allowed"
            >
             {checkingAvailability ? 'Checking Availability...' : 'Continue to Guest Information'}
            </button>
           </div>
          </>
        );

      case 2:
```

```
      return (
        <>
          {/* Step 2: Guest Information */}
          <div className="bg-white p-6 rounded-lg shadow-md mb-6">
            <h3 className="text-xl font-bold mb-6 text-gray-800 pb-2 border-b">Guest
Information</h3>
            <div className="grid md:grid-cols-2 gap-6">
              <div>
                <label className="block text-gray-700 mb-2">First Name <span className="text-
red-600">*</span></label>
                <input
                  type="text"
                  name="firstName"
                  value={formData.firstName}
                  onChange={handleChange}
                  required
                  className="w-full px-4 py-3 border border-gray-300 rounded-lg focus:ring
focus:ring-indigo-200 focus:border-indigo-600 outline-none transition"
                />
              </div>
              <div>
                <label className="block text-gray-700 mb-2">Last Name <span className="text-
red-600">*</span></label>
                <input
                  type="text"
                  name="lastName"
                  value={formData.lastName}
                  onChange={handleChange}
                  required
                  className="w-full px-4 py-3 border border-gray-300 rounded-lg focus:ring
focus:ring-indigo-200 focus:border-indigo-600 outline-none transition"
                />
              </div>
              <div>
                <label className="block text-gray-700 mb-2">Email Address <span
className="text-red-600">*</span></label>
                <input
                  type="email"
                  name="email"
                  value={formData.email}
                  onChange={handleChange}
                  required
                  className="w-full px-4 py-3 border border-gray-300 rounded-lg focus:ring
focus:ring-indigo-200 focus:border-indigo-600 outline-none transition"
                />
              </div>
              <div>
```

```
            <label className="block text-gray-700 mb-2">Phone Number <span
className="text-red-600">*</span></label>
            <input
              type="tel"
              name="phone"
              value={formData.phone}
              onChange={handleChange}
              required
              className="w-full px-4 py-3 border border-gray-300 rounded-lg focus:ring
focus:ring-indigo-200 focus:border-indigo-600 outline-none transition"
            />
           </div>
          </div>
          <div className="mt-6">
           <label className="block text-gray-700 mb-2">Special Requests (Optional)</label>
           <textarea
            name="specialRequests"
            value={formData.specialRequests}
            onChange={handleChange}
            rows="4"
            className="w-full px-4 py-3 border border-gray-300 rounded-lg focus:ring
focus:ring-indigo-200 focus:border-indigo-600 outline-none transition"
           ></textarea>
          </div>
         </div>

         <div className="flex justify-between">
          <button
           type="button"
           onClick={prevStep}
           className="bg-gray-200 text-gray-800 py-3 px-8 rounded-lg font-medium
hover:bg-gray-300 transition-colors"
          >
           Back
          </button>
          <button
           type="submit"
           disabled={loading}
           className="bg-indigo-600 text-white py-3 px-8 rounded-lg font-medium hover:bg-
indigo-700 transition-colors disabled:bg-indigo-400 disabled:cursor-not-allowed"
          >
           {loading ? 'Processing...' : 'Complete Booking'}
          </button>
         </div>
        </>
      );
```

```
    case 3:
      return (
        <div className="bg-white p-8 rounded-lg shadow-md text-center">
          <div className="w-20 h-20 bg-green-100 rounded-full flex items-center justify-
center mx-auto mb-6">
            <svg className="w-10 h-10 text-green-600" fill="none" stroke="currentColor"
viewBox="0 0 24 24">
              <path strokeLinecap="round" strokeLinejoin="round" strokeWidth={2} d="M5 13l4
4L19 7" />
            </svg>
          </div>
          <h3 className="text-2xl font-bold text-gray-800 mb-4">Booking Confirmed!</h3>
          <p className="text-gray-600 mb-6">Thank you for choosing Zorp Hotel. Your
booking has been confirmed.</p>
          <p className="text-gray-600 mb-2">Booking Details:</p>
          <div className="bg-gray-50 p-4 rounded-lg mb-6 inline-block text-left">
            <div className="aspect-video bg-gray-200 rounded-lg overflow-hidden mb-4">
              <img src={selectedRoom.images?.
                [0] ||
"https://images.pexels.com/photos/271624/pexels-photo-271624.jpeg"}
                alt={selectedRoom.type}
                className="w-full h-full object-cover"
                onError={handleImageError}
              />
            </div>
            <p className="text-gray-700"><span className="font-semibold">Room:</span>
{selectedRoom.type}</p>
            <p  className="text-gray-700"><span  className="font-semibold">Check-in:</span>
{new Date(formData.startDate).toLocaleDateString()}</p>
            <p className="text-gray-700"><span className="font-semibold">Check-
out:</span> {new Date(formData.endDate).toLocaleDateString()}</p>
            <p className="text-gray-700"><span className="font-semibold">Total:</span> Rs.
{total}</p>
          </div>
          <div className="space-y-3">
            {formData.startDate && formData.endDate && (
              <div className="flex justify-between text-gray-600">
                <span>Stay Duration:</span>
                <span>{stayDuration} {stayDuration === 1 ? 'Night' : 'Nights'}</span>
              </div>
            )}

            <div className="flex justify-between text-gray-600">
              <span>Room Charge:</span>
              <span>Rs.{roomCharge || selectedRoom.price || 0}</span>
            </div>
```

```
            <div className="flex justify-between text-gray-600">
              <span>Taxes & Fees:</span>
              <span>Rs.{taxes || 0}</span>
            </div>

            <div className="pt-3 border-t mt-3">
              <div className="flex justify-between font-bold text-lg">
                <span>Total:</span>
                <span>Rs.{total || 0}</span>
              </div>
              {stayDuration > 0 && (
                <p className="text-sm text-gray-500 mt-2">* Price shown is for {stayDuration}
{stayDuration === 1 ? 'night' : 'nights'}</p>
              )}
            </div>
          </div>
          <p className="text-gray-600 mb-6">We've sent a confirmation email to
{formData.email} with all the details.</p>
          <button
            onClick={() => navigate('/dashboard')}
            className="bg-indigo-600 text-white py-3 px-8 rounded-lg font-medium hover:bg-
indigo-700 transition-colors"
          >
            View My Bookings
          </button>
        </div>
      );

    default:
      return null;
  }
};

 return (
  <>
    <Nav />
    <ToastContainer position="top-right" autoClose={5000} />

    {/* Hero Section */}
    <section className="pt-32 pb-20 bg-gradient-to-b from-purple-900 to-indigo-900">
      <div className="container mx-auto px-4">
        <div className="max-w-4xl mx-auto text-center">
          <h1 className="text-4xl md:text-5xl font-serif font-bold text-white mb-4">Book Your
<span className="text-amber-400">Perfect Stay</span></h1>
          <p className="text-xl text-gray-300 max-w-2xl mx-auto">Select from our premium
rooms and suites for an unforgettable experience at our luxury hotel.</p>
        </div>
```

```
      </div>
    </section>

    {/* Booking Section */}
    <section className="py-16 bg-gradient-to-b from-indigo-900 to-purple-900">
      <div className="container mx-auto px-4">
        <div className="max-w-6xl mx-auto">
          <div className="flex flex-col lg:flex-row gap-12">

            {/* Form Section */}
            <div className="lg:w-7/12">
              <h2 className="text-3xl font-serif font-bold text-gray-800 mb-8">
                {currentStep === 1 && 'Select Your Stay Details'}
                {currentStep === 2 && 'Complete Your Information'}
                {currentStep === 3 && 'Booking Confirmed'}
              </h2>

              <form onSubmit={handleSubmit} className="space-y-8">
                {renderStepContent()}
              </form>
            </div>

            {/* Booking Summary */}
            {currentStep < 3 && (
              <div className="lg:w-5/12">
                <div className="bg-white rounded-lg shadow-md p-6 sticky top-24">
                  <h2 className="text-2xl font-serif font-bold text-gray-800 mb-6 pb-2 border-
b">Booking Summary</h2>

                  {selectedRoom && Object.keys(selectedRoom).length > 0 ? (
                    <div className="mb-6">
                      <div className="aspect-video bg-gray-200 rounded-lg overflow-hidden mb-
4">
                        <img src={selectedRoom.images?.
                          [0] ||
"https://images.pexels.com/photos/271624/pexels-photo-271624.jpeg"}
                          alt={selectedRoom.type}
                          className="w-full h-full object-cover"
                          onError={handleImageError}
                        />
                      </div>
                      <h3 className="text-xl font-bold text-gray-800 mb-2">{selectedRoom.type ||
'Selected Room'}</h3>
                      <p className="text-gray-600 mb-4">{selectedRoom.description || 'Room
details loading...'}</p>

                      {selectedRoom.amenities && selectedRoom.amenities.length > 0 && (
```

```
                <>
                  <h4 className="font-bold text-gray-700 mb-2">Room Amenities:</h4>
                  <ul className="space-y-1 mb-6">
                   {selectedRoom.amenities.map((amenity, index) => (
                     <li key={index} className="flex items-center text-gray-600">
                       <svg className="w-4 h-4 mr-2 text-indigo-500" fill="currentColor"
viewBox="0 0 20 20">
                         <path fillRule="evenodd" d="M16.707 5.293a1 1 0 010 1.414l-8 8a1 1 0
01-1.414 0l-4-4a1 1 0 011.414-1.414L8 12.586l7.293-7.293a1 1 0 011.414 0z"
clipRule="evenodd" />
                       </svg>
                       {amenity}
                     </li>
                   ))}
                  </ul>
                 </>
                )}
               </div>
              ) : (
               <div className="mb-6 text-gray-500 text-center py-4">
                {loading ? 'Loading room information...' : 'Please select a room to continue'}
               </div>
              )}

              <div className="space-y-3">
               {formData.startDate && formData.endDate && (
                 <div className="flex justify-between text-gray-600">
                   <span>Stay Duration:</span>
                   <span>{stayDuration} {stayDuration === 1 ? 'Night' : 'Nights'}</span>
                 </div>
               )}

               <div className="flex justify-between text-gray-600">
                 <span>Room Charge:</span>
                 <span>Rs.{roomCharge || selectedRoom.price || 0}</span>
               </div>

               <div className="flex justify-between text-gray-600">
                 <span>Taxes & Fees:</span>
                 <span>Rs.{taxes || 0}</span>
               </div>

               <div className="pt-3 border-t mt-3">
                 <div className="flex justify-between font-bold text-lg">
                   <span>Total:</span>
                   <span>Rs.{total || 0}</span>
                 </div>
```

```
                {stayDuration > 0 && (
                  <p className="text-sm text-gray-500 mt-2">* Price shown is for
{stayDuration} {stayDuration === 1 ? 'night' : 'nights'}</p>
                )}
               </div>
              </div>
             </div>
            </div>
           )}
          </div>
         </div>
        </section>
   </>
  );
}

export default Booking;

import { useState } from 'react';
import { Link } from 'react-router-dom';
import Nav from "../components/Nav";

const Contact = () => {
  const [formData, setFormData] =
    useState({ name: '',
    email: '',
    subject: '',
    message: ''
  });

  const [isSubmitting, setIsSubmitting] = useState(false);
  const [submitStatus, setSubmitStatus] = useState(null);

  const handleChange = (e) =>
    { const { name, value } =
    e.target; setFormData(prev =>
    ({
      ...prev,
     [name]: value
    }));
  };

  const handleSubmit = async (e) => {
    e.preventDefault();
    setIsSubmitting(true);

    // Simulating API call
```

```
    try {
      // Replace with actual API call when backend endpoint is ready
      // await api.post('/contact', formData);

      // Simulate API delay
      await new Promise(resolve => setTimeout(resolve, 1000));

      setSubmitStatus({ succ
        ess: true,
        message: 'Your message has been sent successfully! We\'ll get back to you soon.'
      });

      // Clear form
      setFormData({
      name: '',
        email: '',
        subject: '',
        message: ''
      });
    } catch (error) {
      setSubmitStatus({ succ
      ess: false,
        message: error.message || 'Something went wrong. Please try again later.'
      });
    } finally {
      setIsSubmitting(false);
    }
  };

  return (
    <>
      <Nav />

      {/* Hero Section */}
      <section className="pt-32 pb-20 bg-indigo-50">
        <div className="container mx-auto px-4">
          <div className="max-w-3xl mx-auto text-center">
            <h1 className="text-4xl md:text-5xl font-serif font-bold text-gray-800 mb-
6">Contact Us</h1>
            <p className="text-xl text-gray-600 mb-8">Have questions or need assistance?
Reach out to our friendly team.</p>
          </div>
        </div>
      </section>

      {/* Contact Form Section */}
      <section className="py-16">
```

```
      <div className="container mx-auto px-4">
        <div className="max-w-6xl mx-auto">
          <div className="flex flex-col lg:flex-row gap-12">

            {/* Contact Information */}
            <div className="lg:w-5/12">
              <h2 className="text-3xl font-serif font-bold text-gray-800 mb-8">Get in
Touch</h2>

              <div className="bg-white p-6 rounded-lg shadow-md mb-8">
                <div className="space-y-6">
                  <div className="flex items-start">
                    <div className="w-12 h-12 bg-indigo-100 rounded-full flex items-center justify-
center mr-4 flex-shrink-0">
                      <svg className="w-6 h-6 text-indigo-600" fill="none" stroke="currentColor"
viewBox="0 0 24 24" xmlns="http://www.w3.org/2000/svg">
                        <path strokeLinecap="round" strokeLinejoin="round" strokeWidth={2}
d="M17.657 16.657L13.414 20.9a1.998 1.998 0 01-2.827 0l-4.244-4.243a8 8 0 1111.314 0z"
/>
                        <path strokeLinecap="round" strokeLinejoin="round" strokeWidth={2}
d="M15 11a3 3 0 11-6 0 3 3 0 016 0z" />
                      </svg>
                    </div>
                    <div>
                      <h3 className="text-lg font-bold text-gray-800 mb-1">Address</h3>
                      <p className="text-gray-600">123 Dubai Street, Dubai main road, Dubai</p>
                    </div>
                  </div>

                  <div className="flex items-start">
                    <div className="w-12 h-12 bg-indigo-100 rounded-full flex items-center justify-
center mr-4 flex-shrink-0">
                      <svg className="w-6 h-6 text-indigo-600" fill="none" stroke="currentColor"
viewBox="0 0 24 24" xmlns="http://www.w3.org/2000/svg">
                        <path strokeLinecap="round" strokeLinejoin="round" strokeWidth={2} d="M3
5a2 2 0 012-2h3.28a1 1 0 01.948.684l1.498 4.493a1 1 0 01-.502 1.21l-2.257 1.13a11.042
11.042 0 005.516 5.516l1.13-2.257a1 1 0 011.21-.502l4.493 1.498a1 1 0 01.684.949V19a2 2
0 01-2 2h-1C9.716 21 3 14.284 3 6V5z" />
                      </svg>
                    </div>
                    <div>
                      <h3 className="text-lg font-bold text-gray-800 mb-1">Phone</h3>
                      <p className="text-gray-600">+91 96776 05417</p>
                    </div>
                  </div>

                  <div className="flex items-start">
```

```
                <div className="w-12 h-12 bg-indigo-100 rounded-full flex items-center justify-
center mr-4 flex-shrink-0">
                  <svg className="w-6 h-6 text-indigo-600" fill="none" stroke="currentColor"
viewBox="0 0 24 24" xmlns="http://www.w3.org/2000/svg">
                    <path strokeLinecap="round" strokeLinejoin="round" strokeWidth={2} d="M3
8l7.89 5.26a2 2 0 002.22 0L21 8M5 19h14a2 2 0 002-2V7a2 2 0 00-2-2H5a2 2 0 00-2 2v10a2
2 0 002 2z" />
                  </svg>
                </div>
                <div>
                  <h3 className="text-lg font-bold text-gray-800 mb-1">Email</h3>
                  <p className="text-gray-600">raghulkannan005@gmail.com</p>
                </div>
              </div>
            </div>
          </div>

          <div className="bg-white p-6 rounded-lg shadow-md">
            <h3 className="text-xl font-bold text-gray-800 mb-4">Business Hours</h3>
            <ul className="space-y-3">
              <li className="flex justify-between">
                <span className="text-gray-600">Monday - Friday:</span>
                <span className="font-medium">9:00 AM - 10:00 PM</span>
              </li>
              <li className="flex justify-between">
                <span className="text-gray-600">Saturday:</span>
                <span className="font-medium">10:00 AM - 11:00 PM</span>
              </li>
              <li className="flex justify-between">
                <span className="text-gray-600">Sunday:</span>
                <span className="font-medium">10:00 AM - 9:00 PM</span>
              </li>
            </ul>
          </div>
        </div>

        {/* Contact Form */}
        <div className="lg:w-7/12">
          <div className="bg-white p-8 rounded-lg shadow-md">
            <h3 className="text-2xl font-serif font-bold text-gray-800 mb-6">Send Us a
Message</h3>

            {submitStatus && (
              <div className={`mb-6 p-4 rounded-md ${submitStatus.success ? 'bg-green-50
text-green-700' : 'bg-red-50 text-red-700'}`}>
                {submitStatus.message}
              </div>
```

```
        )}

        <form onSubmit={handleSubmit} className="space-y-6">
          <div className="grid md:grid-cols-2 gap-6">
            <div>
              <label className="block text-gray-700 mb-2">Your Name</label>
              <input
                type="text"
                name="name"
                value={formData.name}
                onChange={handleChange}
                required
                className="w-full px-4 py-3 border border-gray-300 rounded-lg focus:ring
focus:ring-indigo-200 focus:border-indigo-600 outline-none transition"
              />
            </div>
            <div>
              <label className="block text-gray-700 mb-2">Your Email</label>
              <input
                type="email"
                name="email"
                value={formData.email}
                onChange={handleChange}
                required
                className="w-full px-4 py-3 border border-gray-300 rounded-lg focus:ring
focus:ring-indigo-200 focus:border-indigo-600 outline-none transition"
              />
            </div>
          </div>

          <div>
            <label className="block text-gray-700 mb-2">Subject</label>
            <input
              type="text"
              name="subject"
              value={formData.subject}
              onChange={handleChange}
              required
              className="w-full px-4 py-3 border border-gray-300 rounded-lg focus:ring
focus:ring-indigo-200 focus:border-indigo-600 outline-none transition"
            />
          </div>

          <div>
            <label className="block text-gray-700 mb-2">Your Message</label>
            <textarea
              name="message"
```

```
                value={formData.message}
                onChange={handleChange}
                rows="6"
                required
                className="w-full px-4 py-3 border border-gray-300 rounded-lg focus:ring
focus:ring-indigo-200 focus:border-indigo-600 outline-none transition"
              ></textarea>
            </div>

            <button
              type="submit"
              disabled={isSubmitting}
              className="w-full bg-indigo-600 text-white py-3 px-6 rounded-lg font-bold
text-lg hover:bg-indigo-700 transition-colors disabled:bg-indigo-400 disabled:cursor-not-
allowed"
            >
              {isSubmitting ? 'Sending...' : 'Send Message'}
            </button>
          </form>
        </div>
      </div>
    </div>
  </section>

  {/* Map Section */}
  <section className="py-16 bg-gray-50">
    <div className="container mx-auto px-4">
      <div className="max-w-6xl mx-auto">
        <h2 className="text-3xl font-serif font-bold text-center text-gray-800 mb-8">Our
Location</h2>
        <div className="bg-white p-2 rounded-lg shadow-md overflow-hidden h-96">
          <iframe

src="https://www.google.com/maps/embed?pb=!1m18!1m12!1m3!1d3608.467755179814
7!2d55.27132507613131!3d25.24618268278219!2m3!1f0!2f0!3f0!3m2!1i1024!2i768!4f13.
1!3m3!1m2!1s0x3e5f42eab00f347b%3A0xb27ba145b3349067!2sBurj%20Khalifa!5e0!3m2!
1sen!2sin!4v1699959638324!5m2!1sen!2sin"
            width="100%"
            height="100%"
            style={{ border: 0 }}
            allowFullScreen=""
            loading="lazy"
            referrerPolicy="no-referrer-when-downgrade"
          ></iframe>
        </div>
```

```
        </div>
       </div>
     </section>

     {/* CTA Section */}
     <section className="py-16 bg-indigo-600">
       <div className="container mx-auto px-4">
         <div className="max-w-3xl mx-auto text-center">
           <h2 className="text-3xl font-serif font-bold text-white mb-6">Ready to Experience
Luxury?</h2>
           <p className="text-xl text-indigo-100 mb-8">Book your stay at Zorp Hotel and
indulge in unparalleled luxury and service.</p>
           <Link to="/booking" className="inline-block bg-white text-indigo-600 px-8 py-4
rounded-lg font-bold hover:bg-gray-100 transition-colors">Book Now</Link>
         </div>
       </div>
     </section>
   </>
  );
}

export default Contact;

import { useState, useEffect } from 'react';
import { Link } from 'react-router-dom';
import Nav from "../components/Nav";
import ChangePasswordModal from "../components/ChangePasswordModal";
import { getBookingHistory, cancelBooking } from '../services/bookingService';
import { getCurrentUser, logoutUser } from '../services/authService';
import { toast, ToastContainer } from 'react-toastify';
import 'react-toastify/dist/ReactToastify.css';

const Dashboard = () => {
  const [activeTab, setActiveTab] = useState('bookings');
  const [bookings, setBookings] = useState([]);
  const [loading, setLoading] = useState(true);
  const [user, setUser] = useState(null);
  const [isChangePasswordModalOpen, setIsChangePasswordModalOpen] = useState(false);

  useEffect(() => {
    const user = getCurrentUser();
    setUser(user);

    fetchBookings();
  }, []);

  const fetchBookings = async () => {
```

```
  try {
    setLoading(true);
    const bookingHistory = await getBookingHistory();
    setBookings(bookingHistory);
  } catch (error) {
    console.error('Error fetching bookings:', error);
    toast.error('Failed to load your bookings');
  } finally {
    setLoading(false);
  }
};

const handleCancelBooking = async (bookingId) => {
  if (window.confirm('Are you sure you want to cancel this booking?')) {
    try {
      await cancelBooking(bookingId);
      toast.success('Booking cancelled successfully');
      // Refresh bookings
      fetchBookings();
    } catch (error) {
      console.error('Error cancelling booking:', error);
      toast.error(error.message || 'Failed to cancel booking');
    }
  }
};

const handleLogout = () => {
  logoutUser();
  window.location.href = '/';
};

const formatDate = (dateString) => {
  const options = { year: 'numeric', month: 'long', day: 'numeric' };
  return new Date(dateString).toLocaleDateString(undefined, options);
};

return (
  <>
    <Nav />
    <ToastContainer position="top-right" autoClose={5000} />

    {/* Change Password Modal */}
    <ChangePasswordModal
      isOpen={isChangePasswordModalOpen}
      onClose={() => setIsChangePasswordModalOpen(false)}
    />
```

```
    {/* Hero Section */}
    <section className="pt-32 pb-20 bg-gradient-to-b from-purple-900 to-indigo-900">
      <div className="container mx-auto px-4">
        <div className="max-w-4xl mx-auto">
          <div className="flex flex-col md:flex-row items-center justify-between mb-8 gap-4">
            <div>
              <h1 className="text-4xl font-serif font-bold text-white mb-2">My <span
className="text-amber-400">Dashboard</span></h1>
              <p className="text-xl text-gray-300">Welcome back, <span className="text-
amber-300 font-medium">{user?.username || 'Guest'}</span></p>
            </div>
            <button
              onClick={handleLogout}
              className="bg-gradient-to-r from-amber-500 to-amber-600 text-indigo-900 py-2
px-6 rounded-lg font-medium hover:from-amber-400 hover:to-amber-500 transition-all
shadow-lg transform hover:-translate-y-1 uppercase tracking-wider"
            >
              Log Out
            </button>
          </div>

          {/* Tabs */}
          <div className="bg-white/10 backdrop-blur-sm rounded-lg overflow-hidden
shadow-xl border border-white/10">
            <div className="flex border-b border-white/10">
              <button
                className={`px-6 py-3 text-lg font-medium transition-all ${activeTab ===
'bookings' ? 'border-b-2 border-amber-400 text-amber-400' : 'text-gray-300 hover:text-
white'}`}
                onClick={() => setActiveTab('bookings')}
              >
                My Bookings
              </button>
              <button
                className={`px-6 py-3 text-lg font-medium transition-all ${activeTab === 'profile'
? 'border-b-2 border-amber-400 text-amber-400' : 'text-gray-300 hover:text-white'}`}
                onClick={() => setActiveTab('profile')}
              >
                Profile
              </button>
            </div>
          </div>
        </div>
      </div>
    </section>

    {/* Dashboard Content */}
```

```
    <section className="py-16 bg-gradient-to-b from-indigo-900 to-purple-900">
      <div className="container mx-auto px-4">
        <div className="max-w-4xl mx-auto">

          {/* Bookings Tab */}
          {activeTab === 'bookings' && (
            <div>
              <div className="flex flex-col sm:flex-row justify-between items-center mb-8 gap-
4">
                <h2 className="text-2xl font-serif font-bold text-white">Your <span
className="text-amber-400">Bookings</span></h2>
                <Link
                  to="/booking"
                  className="bg-gradient-to-r from-amber-500 to-amber-600 text-indigo-900 py-
2 px-6 rounded-lg font-medium hover:from-amber-400 hover:to-amber-500 transition-all
shadow-lg transform hover:-translate-y-1 uppercase tracking-wider text-sm"
                >
                  Book Another Stay
                </Link>
              </div>

              {loading ? (
                <div className="flex justify-center py-8">
                  <div className="animate-spin rounded-full h-12 w-12 border-b-2 border-amber-
400"></div>
                </div>
              ) : bookings.length > 0 ? (
                <div className="space-y-6">
                  {bookings.map(booking => (
                    <div key={booking._id} className="bg-white/10 backdrop-blur-sm p-6
rounded-lg shadow-xl border border-white/10 hover:shadow-2xl transition-all transform
hover:-translate-y-1">
                      <div className="flex flex-col md:flex-row justify-between">
                        <div>
                          <h3 className="text-xl font-bold text-white mb-
2">{booking.room.type}</h3>
                          <div className="flex items-center text-gray-300 mb-4">
                            <svg className="w-5 h-5 mr-2 text-amber-400" fill="none"
stroke="currentColor" viewBox="0 0 24 24">
                              <path strokeLinecap="round" strokeLinejoin="round" strokeWidth={2}
d="M8 7V3m8 4V3m-9 8h10M5 21h14a2 2 0 002-2V7a2 2 0 00-2-2H5a2 2 0 00-2 2v12a2 2 0
002 2z" />
                            </svg>
                            <span>{formatDate(booking.startDate)} -
{formatDate(booking.endDate)}</span>
                          </div>
                          <div className="flex items-center text-gray-300 mb-2">
```

```
                    <svg className="w-5 h-5 mr-2 text-amber-400" fill="none"
stroke="currentColor" viewBox="0 0 24 24">
                      <path strokeLinecap="round" strokeLinejoin="round" strokeWidth={2}
d="M17 9V7a2 2 0 00-2-2H5a2 2 0 00-2 2v6a2 2 0 002 2h2m2 4h10a2 2 0 002-2v-6a2 2 0 00-
2-2H9a2 2 0 00-2 2v6a2 2 0 002 2z" />
                    </svg>
                    <span>Booking ID: {booking._id.substring(0, 8)}</span>
                  </div>
                  <div className="flex items-center text-gray-300">
                    <svg className="w-5 h-5 mr-2 text-amber-400" fill="none"
stroke="currentColor" viewBox="0 0 24 24">
                      <path strokeLinecap="round" strokeLinejoin="round" strokeWidth={2}
d="M9 12l2 2 4-4m6 2a9 9 0 11-18 0 9 9 0 0118 0z" />
                    </svg>
                    <span>Status: <span className={`font-medium ${booking.status ===
'booked' ? 'text-green-400' : 'text-red-400'}`}>{booking.status.charAt(0).toUpperCase() +
booking.status.slice(1)}</span></span>
                  </div>
                </div>

                <div className="mt-4 md:mt-0">
                  {booking.status === 'booked' && (
                    <button
                      onClick={() => handleCancelBooking(booking._id)}
                      className="px-4 py-2 border border-red-400 text-red-400 rounded-md
hover:bg-red-400/10 transition-colors"
                    >
                      Cancel Booking
                    </button>
                  )}
                </div>
              </div>
            </div>
          ))}
        </div>
      ) : (
        <div className="bg-white/10 backdrop-blur-sm p-8 rounded-lg shadow-xl border
border-white/10 text-center">
          <div className="text-amber-400 mb-4">
            <svg className="w-16 h-16 mx-auto" fill="none" stroke="currentColor"
viewBox="0 0 24 24">
              <path strokeLinecap="round" strokeLinejoin="round" strokeWidth={2} d="M9
5H7a2 2 0 00-2 2v12a2 2 0 002 2h10a2 2 0 002-2V7a2 2 0 00-2-2h-2M9 5a2 2 0 002 2h2a2 2
0 002-2M9 5a2 2 0 012-2h2a2 2 0 012 2" />
            </svg>
          </div>
          <h3 className="text-xl font-bold text-white mb-2">No Bookings Yet</h3>
```

```
            <p className="text-gray-300 mb-6">You haven't made any bookings yet. Ready to plan your stay?</p>
            <Link
              to="/booking"
              className="bg-gradient-to-r from-amber-500 to-amber-600 text-indigo-900 py-2 px-6 rounded-lg font-medium hover:from-amber-400 hover:to-amber-500 transition-all shadow-lg transform hover:-translate-y-1 uppercase tracking-wider"
            >
              Book Your First Stay
            </Link>
          </div>
        )}
      </div>
    )}

    {/* Profile Tab */}
    {activeTab === 'profile' && (
      <div>
        <h2 className="text-2xl font-serif font-bold text-white mb-8">Your <span className="text-amber-400">Profile</span></h2>

        <div className="bg-white/10 backdrop-blur-sm p-6 rounded-lg shadow-xl border border-white/10">
          <div className="grid md:grid-cols-2 gap-6">
            <div>
              <h3 className="text-amber-400 mb-1 uppercase tracking-wider text-sm font-medium">Username</h3>
              <p className="text-xl font-medium text-white">{user?.username || 'N/A'}</p>
            </div>
            <div>
              <h3 className="text-amber-400 mb-1 uppercase tracking-wider text-sm font-medium">Email</h3>
              <p className="text-xl font-medium text-white">{user?.email || 'N/A'}</p>
            </div>
            <div>
              <h3 className="text-amber-400 mb-1 uppercase tracking-wider text-sm font-medium">Account Type</h3>
              <p className="text-xl font-medium text-white capitalize">{user?.role || 'user'}</p>
            </div>
            <div>
              <h3 className="text-amber-400 mb-1 uppercase tracking-wider text-sm font-medium">Member Since</h3>
              <p className="text-xl font-medium text-white">{user?.createdAt ? formatDate(user.createdAt) : 'N/A'}</p>
            </div>
          </div>
```

```
            <div className="mt-8 pt-6 border-t border-white/10">
              <h3 className="text-lg font-bold text-white mb-4">Account Settings</h3>
              <div className="space-y-4">
                <button className="flex items-center text-amber-400 hover:text-amber-300
transition-colors group">
                  <svg className="w-5 h-5 mr-2 group-hover:scale-110 transition-transform"
fill="none" stroke="currentColor" viewBox="0 0 24 24">
                    <path strokeLinecap="round" strokeLinejoin="round" strokeWidth={2}
d="M15.232 5.232l3.536 3.536m-2.036-5.036a2.5 2.5 0 113.536 3.536L6.5 21.036H3v-
3.572L16.732 3.732z" />
                  </svg>
                  Edit Profile
                </button>
                <button
                  onClick={() => setIsChangePasswordModalOpen(true)}
                  className="flex items-center text-amber-400 hover:text-amber-300
transition-colors group"
                >
                  <svg className="w-5 h-5 mr-2 group-hover:scale-110 transition-transform"
fill="none" stroke="currentColor" viewBox="0 0 24 24">
                    <path strokeLinecap="round" strokeLinejoin="round" strokeWidth={2}
d="M12 15v2m-6 4h12a2 2 0 002-2v-6a2 2 0 00-2-2H6a2 2 0 00-2 2v6a2 2 0 002 2zm10-
10V7a4 4 0 00-8 0v4h8z" />
                  </svg>
                  Change Password
                </button>
                <button
                  onClick={handleLogout}
                  className="flex items-center text-red-400 hover:text-red-300 transition-
colors group"
                >
                  <svg className="w-5 h-5 mr-2 group-hover:scale-110 transition-transform"
fill="none" stroke="currentColor" viewBox="0 0 24 24">
                    <path strokeLinecap="round" strokeLinejoin="round" strokeWidth={2}
d="M17 16l4-4m0 0l-4-4m4 4H7m6 4v1a3 3 0 01-3 3H6a3 3 0 01-3-3V7a3 3 0 013-3h4a3 3 0
013 3v1" />
                  </svg>
                  Log Out
                </button>
              </div>
            </div>
          </div>
        )}

      </div>
```

```
      </div>
    </section>
  </>
 );
};

export default Dashboard;

import { useState, useEffect } from 'react';
import { Link } from 'react-router-dom';
import Nav from "../components/Nav";
import { getFeaturedRooms } from '../services/roomService';
import { toast, ToastContainer } from 'react-toastify';
import 'react-toastify/dist/ReactToastify.css';

const Landing = () => {
 const [featuredRooms, setFeaturedRooms] = useState([]);
 const [loading, setLoading] = useState(true);

 useEffect(() => {
  const fetchRooms = async () =>
   { try {
    setLoading(true);
    const roomsData = await getFeaturedRooms();
    setFeaturedRooms(roomsData);
   } catch (error) {
    console.error('Error fetching rooms:', error);
    toast.error('Failed to load rooms. Please try again later.');
   } finally {
    setLoading(false);
   }
  };

  fetchRooms();
 }, []);

 // Default room data if no featured rooms are available
 const defaultRooms = [
  {
   _id: 'default1',
   type: 'Deluxe Room',
   price: 199,
   description: 'Spacious room with modern amenities and a beautiful view.',
   images: ['https://images.pexels.com/photos/271624/pexels-photo-271624.jpeg'],
   capacity: 2
  },
  {
```

```
    _id: 'default2',
    type: 'Executive Suite',
    price: 299,
    description: 'Luxurious suite with separate living area and premium amenities.',
    images: ['https://images.pexels.com/photos/1457842/pexels-photo-1457842.jpeg'],
    capacity: 2
  },
  {
    _id: 'default3',
    type: 'Family Room',
    price: 249,
    description: 'Comfortable room designed for families with extra space and amenities.',
    images: ['https://images.pexels.com/photos/2507016/pexels-photo-2507016.jpeg'],
    capacity: 4
  },
  {
    _id: 'default4',
    type: 'Presidential Suite',
    price: 499,
    description: 'Our most luxurious accommodation with exclusive services and amenities.',
    images: ['https://images.pexels.com/photos/164595/pexels-photo-164595.jpeg'],
    capacity: 2
  }
];

// Use featured rooms from API if available, otherwise use default rooms
const displayRooms = featuredRooms.length > 0 ? featuredRooms : defaultRooms;

return (
  <>
    <Nav />
    <ToastContainer position="top-right" autoClose={5000} />

    <section className="relative h-screen flex items-center">
      <div className="absolute inset-0 bg-black opacity-50 z-10"></div>
      <div className="absolute inset-0 bg-[url('https://images.unsplash.com/photo-1566073771259-6a8506099945?q=80&w=1920')] bg-cover bg-center"></div>
      <div className="absolute inset-0 bg-gradient-to-b from-transparent via-transparent to-purple-900/70 z-10"></div>
      <div className="container mx-auto px-4 relative z-20">
        <div className="max-w-3xl">
          <h1 className="text-5xl md:text-6xl font-serif font-bold text-white mb-4 leading-tight drop-shadow-lg">
            Experience <span className="text-amber-400">Luxury</span> at Zorp Hotel
          </h1>
          <p className="text-xl text-white mb-8 drop-shadow-md">Where comfort meets elegance. Enjoy our premium amenities and world-class service.</p>
```

```
          <div className="flex flex-col sm:flex-row space-y-4 sm:space-y-0 sm:space-x-4">
            <Link to="/booking" className="bg-gradient-to-r from-amber-500 to-amber-600
text-indigo-900 px-8 py-4 rounded-md hover:from-amber-400 hover:to-amber-500
transition-all font-medium text-lg shadow-lg transform hover:-translate-y-1 uppercase
tracking-wider">Book Now</Link>
            <Link to="/about" className="bg-white/10 backdrop-blur-sm text-white border
border-white/30 px-8 py-4 rounded-md hover:bg-white/20 transition-all font-medium text-
lg shadow-lg transform hover:-translate-y-1 uppercase tracking-wider">Learn More</Link>
          </div>
        </div>
      </div>
    </section>

    {/* Features Section */}
    <section className="py-20 bg-gradient-to-b from-purple-900 to-indigo-900">
      <div className="container mx-auto px-4">
        <h2 className="text-3xl font-serif font-bold text-center mb-16 text-white">Why
Choose <span className="text-amber-400">Zorp Hotel</span></h2>

        <div className="grid md:grid-cols-3 gap-8">
          {/* Feature 1 */}
          <div className="bg-white/10 backdrop-blur-sm p-8 rounded-lg shadow-xl
hover:shadow-2xl transition-all border border-white/10 transform hover:-translate-y-2
group">
            <div className="w-16 h-16 bg-amber-400 rounded-full flex items-center justify-
center mb-6 mx-auto group-hover:scale-110 transition-transform">
              <svg className="w-8 h-8 text-indigo-900" fill="none" stroke="currentColor"
viewBox="0 0 24 24" xmlns="http://www.w3.org/2000/svg">
                <path strokeLinecap="round" strokeLinejoin="round" strokeWidth={2} d="M5
3v4M3 5h4M6 17v4m-2-2h4m5-16l2.286 6.857L21 12l-5.714 2.143L13 21l-2.286-6.857L5
12l5.714-2.143L13 3z" />
              </svg>
            </div>
            <h3 className="text-xl font-bold mb-3 text-white text-center">Luxury
Experience</h3>
            <p className="text-gray-300 text-center">Indulge in our luxurious accommodations
designed for your ultimate comfort and relaxation.</p>
          </div>

          {/* Feature 2 */}
          <div className="bg-white/10 backdrop-blur-sm p-8 rounded-lg shadow-xl
hover:shadow-2xl transition-all border border-white/10 transform hover:-translate-y-2
group">
            <div className="w-16 h-16 bg-amber-400 rounded-full flex items-center justify-
center mb-6 mx-auto group-hover:scale-110 transition-transform">
              <svg className="w-8 h-8 text-indigo-900" fill="none" stroke="currentColor"
viewBox="0 0 24 24" xmlns="http://www.w3.org/2000/svg">
```

```
              <path strokeLinecap="round" strokeLinejoin="round" strokeWidth={2} d="M12
8v4l3 3m6-3a9 9 0 11-18 0 9 9 0 0118 0z" />
            </svg>
          </div>
          <h3 className="text-xl font-bold mb-3 text-white text-center">24/7 Service</h3>
          <p className="text-gray-300 text-center">Our dedicated staff is always available to
cater to your needs, day or night.</p>
        </div>

        {/* Feature 3 */}
        <div className="bg-white/10 backdrop-blur-sm p-8 rounded-lg shadow-xl
hover:shadow-2xl transition-all border border-white/10 transform hover:-translate-y-2
group">
          <div className="w-16 h-16 bg-amber-400 rounded-full flex items-center justify-
center mb-6 mx-auto group-hover:scale-110 transition-transform">
            <svg className="w-8 h-8 text-indigo-900" fill="none" stroke="currentColor"
viewBox="0 0 24 24" xmlns="http://www.w3.org/2000/svg">
              <path strokeLinecap="round" strokeLinejoin="round" strokeWidth={2} d="M21
15.546c-.523 0-1.046.151-1.5.454a2.704 2.704 0 01-3 0 2.704 2.704 0 00-3 0 2.704 2.704 0
01-3 0 2.704 2.704 0 00-3 0 2.701 2.701 0 00-1.5-.454M9 6v2m3-2v2m3-2v2M9 3h.01M12
3h.01M15 3h.01M21 21v-7a2 2 0 00-2-2H5a2 2 0 00-2 2v7h18zm-3-9v-2a2 2 0 00-2-2H8a2 2
0 00-2 2v2h12z" />
            </svg>
          </div>
          <h3 className="text-xl font-bold mb-3 text-white text-center">Fine Dining</h3>
          <p className="text-gray-300 text-center">Experience exquisite culinary delights
prepared by our world-class chefs.</p>
        </div>
      </div>
    </div>
  </section>

  {/* Rooms Preview Section */}
  <section className="py-20 bg-gradient-to-b from-indigo-900 to-purple-900">
    <div className="container mx-auto px-4">
      <h2 className="text-3xl font-serif font-bold text-center mb-4 text-white">Our <span
className="text-amber-400">Premium Rooms</span></h2>
      <p className="text-center text-gray-300 mb-16 max-w-2xl mx-auto">Choose from
our selection of luxury rooms and suites, each designed to offer you the ultimate comfort
during your stay.</p>

      {loading ? (
        <div className="flex justify-center py-12">
          <div className="animate-spin rounded-full h-16 w-16 border-b-2 border-amber-
400"></div>
        </div>
      ) : (
```

```
        <div className="grid md:grid-cols-2 lg:grid-cols-3 gap-8">
          {displayRooms.map(room => (
            <div key={room._id} className="bg-white/10 backdrop-blur-sm rounded-lg
overflow-hidden shadow-xl hover:shadow-2xl transition-all border border-white/10 group">
              <div className="relative overflow-hidden">
                <img
                  src={room.images?.[0] || "https://images.pexels.com/photos/271624/pexels-
photo-271624.jpeg"}
                  alt={room.type}
                  className="w-full h-64 object-cover transform group-hover:scale-110
transition-transform duration-500"
                />
                <div className="absolute inset-0 bg-gradient-to-t from-purple-900 to-
transparent opacity-70"></div>
              </div>
              <div className="p-6">
                <h3 className="text-xl font-bold mb-2 text-white">{room.type}</h3>
                <p className="text-gray-300 mb-4 line-clamp-2">{room.description}</p>
                <div className="flex justify-between items-center">
                  <span className="text-amber-400 font-bold">₹{room.price} / night</span>
                  <Link to="/booking" className="bg-amber-500 text-indigo-900 px-4 py-2
rounded-md hover:bg-amber-400 transition-colors shadow-md transform hover:-translate-
y-1 hover:shadow-lg">Book Now</Link>
                </div>
              </div>
            </div>
          ))}
        </div>
      )}
    </div>
  </section>

  {/* Testimonial Section */}
  <section className="py-20 bg-gradient-to-b from-purple-900 to-indigo-900">
    <div className="container mx-auto px-4">
      <h2 className="text-3xl font-serif font-bold text-center mb-16 text-white">What Our
<span className="text-amber-400">Guests Say</span></h2>

      <div className="max-w-4xl mx-auto">
        <div className="grid md:grid-cols-2 gap-8">
          <div className="bg-white/10 backdrop-blur-sm p-6 rounded-lg shadow-xl border
border-white/10 transform hover:-translate-y-2 transition-all">
            <div className="flex items-center mb-4">
              <div className="w-16 h-16 rounded-full overflow-hidden mr-4 ring-2 ring-amber-
400">
```

<img
src="https://filmfare.wwmindia.com/content/2023/apr/deepikapadukone11682765623.jpg
" alt="Guest" className="w-full h-full object-cover object-top" />

<h4 className="text-lg font-bold text-white">Deepika Padukone</h4>
Actress

{[...Array(5)].map((_, i) => (
<svg key={i} className="w-5 h-5 text-amber-400" fill="currentColor"
viewBox="0 0 20 20" xmlns="http://www.w3.org/2000/svg">
<path d="M9.049 2.927c.3-.921 1.603-.921 1.902 0l1.07 3.292a1 1 0
00.95.69h3.462c.969 0 1.371 1.24.588 1.81l-2.8 2.034a1 1 0 00-.364 1.118l1.07
3.292c.3.921-.755 1.688-1.54 1.118l-2.8-2.034a1 1 0 00-1.175 0l-2.8 2.034c-.784.57-1.838-
.197-1.539-1.118l1.07-3.292a1 1 0 00-.364-1.118L2.98 8.72c-.783-.57-.38-1.81.588-
1.81h3.461a1 1 0 00.951-.69l1.07-3.292z" />
</svg>
))}

"Between film shoots, Zorp Hotel is my
sanctuary. The staff's discretion and attention to detail is unmatched. The spa treatments
are divine, and I always leave feeling refreshed and rejuvenated. It's truly my home away
from home in the city."

<div className="bg-white/10 backdrop-blur-sm p-6 rounded-lg shadow-xl border
border-white/10 transform hover:-translate-y-2 transition-all">

<div className="w-16 h-16 rounded-full overflow-hidden mr-4 ring-2 ring-amber-
400">
<img
src="https://upload.wikimedia.org/wikipedia/commons/8/86/Aamir_Khan_at_Delhi_Airpor
t.jpg" alt="Guest" className="w-full h-full object-cover object-top" />

<h4 className="text-lg font-bold text-white">Aamir Khan</h4>
Actor & Producer

{[...Array(5)].map((_, i) => (
<svg key={i} className="w-5 h-5 text-amber-400" fill="currentColor"
viewBox="0 0 20 20" xmlns="http://www.w3.org/2000/svg">
<path d="M9.049 2.927c.3-.921 1.603-.921 1.902 0l1.07 3.292a1 1 0
00.95.69h3.462c.969 0 1.371 1.24.588 1.81l-2.8 2.034a1 1 0 00-.364 1.118l1.07
3.292c.3.921-.755 1.688-1.54 1.118l-2.8-2.034a1 1 0 00-1.175 0l-2.8 2.034c-.784.57-1.838-

```
.197-1.539-1.118l1.07-3.292a1 1 0 00-.364-1.118L2.98 8.72c-.783-.57-.38-1.81.588-1.81h3.461a1 1 0 00.951-.69l1.07-3.292z" />
                </svg>
              )))
            </div>
            </div>
            <p className="text-gray-300 italic">"I've stayed at luxury hotels worldwide, but Zorp Hotel stands out for its authentic Indian hospitality with international standards. The culinary offerings are exceptional, and the staff anticipates your needs before you even express them. A true gem!"</p>
          </div>
        </div>
      </div>
    </section>

    {/* Footer */}
    <footer className="bg-gradient-to-r from-purple-950 to-indigo-950 text-white py-16">
      <div className="container mx-auto px-4">
        <div className="grid md:grid-cols-4 gap-8">
          <div>
            <h3 className="text-2xl font-serif font-bold mb-6">Zorp <span className="text-amber-400">Hotel</span></h3>
            <p className="text-gray-300">Where luxury meets comfort. Experience the best hospitality in town.</p>
          </div>
          <div>
            <h4 className="text-lg font-bold mb-6 text-amber-400 uppercase tracking-wider">Quick Links</h4>
            <ul className="space-y-3">
              <li><Link to="/" className="text-gray-300 hover:text-amber-300 transition-colors">Home</Link></li>
              <li><Link to="/about" className="text-gray-300 hover:text-amber-300 transition-colors">About</Link></li>
              <li><Link to="/contact" className="text-gray-300 hover:text-amber-300 transition-colors">Contact</Link></li>
              <li><Link to="/booking" className="text-gray-300 hover:text-amber-300 transition-colors">Book Now</Link></li>
            </ul>
          </div>
          <div>
            <h4 className="text-lg font-bold mb-6 text-amber-400 uppercase tracking-wider">Contact</h4>
            <ul className="space-y-3 text-gray-300">
              <li className="flex items-center">
                <svg className="w-5 h-5 mr-3 text-amber-400" fill="currentColor" viewBox="0 0 20 20" xmlns="http://www.w3.org/2000/svg">
```

<path fillRule="evenodd" d="M5.05 4.05a7 7 0 119.9 9.9L10 18.9l-4.95-4.95a7 7 0 010-9.9zM10 11a2 2 0 100-4 2 2 0 000 4z" clipRule="evenodd" />
</svg>
123 Dubai Street, Dubai main road, Dubai

<li className="flex items-center">
<svg className="w-5 h-5 mr-3 text-amber-400" fill="currentColor" viewBox="0 0 20 20" xmlns="http://www.w3.org/2000/svg">
<path d="M2 3a1 1 0 011-1h2.153a1 1 0 01.986.836l.74 4.435a1 1 0 01-.54 1.06l-1.548.773a11.037 11.037 0 006.105 6.105l.774-1.548a1 1 0 011.059-.54l4.435.74a1 1 0 01.836.986V17a1 1 0 01-1 1h-2C7.82 18 2 12.18 2 5V3z" />
</svg>
+91 96776 05417

<li className="flex items-center">
<svg className="w-5 h-5 mr-3 text-amber-400" fill="currentColor" viewBox="0 0 20 20" xmlns="http://www.w3.org/2000/svg">
<path d="M2.003 5.884L10 9.882l7.997-3.998A2 2 0 0016 4H4a2 2 0 00-1.997 1.884z" />
<path d="M18 8.118l-8 4-8-4V14a2 2 0 002 2h12a2 2 0 002-2V8.118z" />
</svg>
raghulkannan005@gmail.com

<h4 className="text-lg font-bold mb-6 text-amber-400 uppercase tracking-wider">Follow Us</h4>

<svg className="w-5 h-5" fill="currentColor" viewBox="0 0 24 24" xmlns="http://www.w3.org/2000/svg">
<path d="M9 8h-3v4h3v12h5v-12h3.642l.358-4h-4v-1.667c0-.955.192-1.333 1.115-1.333h2.885v-5h-3.808c-3.596 0-5.192 1.583-5.192 4.615v3.385z" />
</svg>

<a href="#" className="w-10 h-10 bg-white/10 rounded-full flex items-center justify-center text-gray-300 hover:bg-amber-400 hover:text-indigo-900 transition-all">
<svg className="w-5 h-5" fill="currentColor" viewBox="0 0 24 24" xmlns="http://www.w3.org/2000/svg">
<path d="M12 2.163c3.204 0 3.584.012 4.85.07 3.252.148 4.771 1.691 4.919 4.919.058 1.265.069 1.645.069 4.849 0 3.205-.012 3.584-.069 4.849-.149 3.225-1.664 4.771-4.919 4.919-1.266.058-1.644.07-4.85.07-3.204 0-3.584-.012-4.849-.07-3.26-.149-4.771-1.699-4.919-4.92-.058-1.265-.07-1.644-.07-4.849 0-3.204.013-3.583.07-4.849.149-3.227 1.664-4.771 4.919-4.919 1.266-.057 1.645-.069 4.849-.069zm0-2.163c-3.259 0-3.667.014-4.947.072-4.358.2-6.78 2.618-6.98 6.98-.059 1.281-.073 1.689-.073 4.948 0 3.259.014

```
3.668.072 4.948.2 4.358 2.618 6.78 6.98 6.98 1.281.058 1.689.072 4.948.072 3.259 0 3.668-
.014 4.948-.072 4.354-.2 6.782-2.618 6.979-6.98.059-1.28.073-1.689.073-4.948 0-3.259-
.014-3.667-.072-4.947-.196-4.354-2.617-6.78-6.979-6.98-1.281-.059-1.69-.073-4.949-
.073zm0 5.838c-3.403 0-6.162 2.759-6.162 6.162s2.759 6.163 6.162 6.163 6.162-2.759
6.162-6.163c0-3.403-2.759-6.162-6.162-6.162zm0 10.162c-2.209 0-4-1.79-4-4 0-2.209
1.791-4 4-4s4 1.791 4 4c0 2.21-1.791 4-4 4zm6.406-11.845c-.796 0-1.441.645-1.441
1.44s.645 1.44 1.441 1.44c.795 0 1.439-.645 1.439-1.44s-.644-1.44-1.439-1.44z" />
              </svg>
             </a>
             <a href="#" className="w-10 h-10 bg-white/10 rounded-full flex items-center
justify-center text-gray-300 hover:bg-amber-400 hover:text-indigo-900 transition-all">
              <svg className="w-5 h-5" fill="currentColor" viewBox="0 0 24 24"
xmlns="http://www.w3.org/2000/svg">
               <path d="M24 4.557c-.883.392-1.832.656-2.828.775 1.017-.609 1.798-1.574
2.165-2.724-.951.564-2.005.974-3.127  1.195-.897-.957-2.178-1.555-3.594-1.555-3.179  0-
5.515 2.966-4.797 6.045-4.091-.205-7.719-2.165-10.148-5.144-1.29 2.213-.669 5.108 1.523
6.574-.806-.026-1.566-.247-2.229-.616-.054 2.281 1.581 4.415 3.949 4.89-.693.188-
1.452.232-2.224.084.626 1.956 2.444 3.379 4.6 3.419-2.07 1.623-4.678 2.348-7.29 2.04
2.179 1.397 4.768 2.212 7.548 2.212 9.142 0 14.307-7.721 13.995-14.646.962-.695 1.797-
1.562 2.457-2.549z" />
              </svg>
             </a>
            </div>
           </div>
          </div>
          <div className="border-t border-white/10 mt-12 pt-8 text-center text-gray-400">
           <p>&copy; {new Date().getFullYear()} Zorp Hotel. All rights reserved.</p>
          </div>
         </div>
        </footer>
       </>
      );
}

export default Landing;

import { useState } from 'react';
import { Link, useNavigate, useLocation } from 'react-router-dom';
import Nav from "../components/Nav";
import { loginUser } from '../services/authService';
import { toast, ToastContainer } from 'react-toastify';
import 'react-toastify/dist/ReactToastify.css';

const Login = () => {
  const navigate = useNavigate();
  const location = useLocation();
  const [isLoading, setIsLoading] = useState(false);
```

```
const [formData, setFormData] =
  useState({ email: '',
  password: ''
});

// Get the redirect path from location state or default to "/"
const from = location.state?.from || "/";

const handleChange = (e) =>
  { const { name, value } =
  e.target; setFormData(prev =>
  ({
    ...prev,
    [name]: value
  }));
};

const handleSubmit = async (e) => {
  e.preventDefault();
  setIsLoading(true);

  try {
    await loginUser(formData);
    toast.success('Login successful!');

    // Redirect after successful login
    setTimeout(() => {
    navigate(from, { replace: true });
    }, 1000);
  } catch (error) {
    console.error('Login error:', error);
    toast.error(error.message || 'Invalid credentials. Please try again.');
  } finally {
    setIsLoading(false);
  }
};

return (
  <>
    <Nav />
    <ToastContainer position="top-right" autoClose={5000} />

    {/* Hero Section */}
    <section className="pt-32 pb-20 bg-gradient-to-b from-purple-900 to-indigo-900">
      <div className="container mx-auto px-4">
        <div className="max-w-3xl mx-auto text-center">
          <h1 className="text-4xl md:text-5xl font-serif font-bold text-white mb-6">
            Login to Your <span className="text-amber-400">Account</span>
```

```
          </h1>
          <p className="text-xl text-gray-300 mb-8">Welcome back! Sign in to access your
bookings and manage your stays.</p>
        </div>
       </div>
     </section>

     {/* Login Form Section */}
     <section className="py-16 bg-gradient-to-b from-indigo-900 to-purple-900">
       <div className="container mx-auto px-4">
         <div className="max-w-md mx-auto bg-white/10 backdrop-blur-sm rounded-lg
shadow-xl p-8 border border-white/10">
          <form onSubmit={handleSubmit} className="space-y-6">
           <div>
             <label className="block text-white mb-2 font-medium">Email Address</label>
             <input
               type="email"
               name="email"
               value={formData.email}
               onChange={handleChange}
               required
               className="w-full px-4 py-3 bg-white/5 border border-white/10 rounded-lg
focus:ring focus:ring-amber-300/50 focus:border-amber-400 outline-none transition text-
white"
               placeholder="Enter your email"
             />
           </div>

           <div>
             <label className="block text-white mb-2 font-medium">Password</label>
             <input
               type="password"
               name="password"
               value={formData.password}
               onChange={handleChange}
               required
               className="w-full px-4 py-3 bg-white/5 border border-white/10 rounded-lg
focus:ring focus:ring-amber-300/50 focus:border-amber-400 outline-none transition text-
white"
               placeholder="Enter your password"
             />
           </div>

           <div className="flex items-center">
             <input
               type="checkbox"
               id="remember"
```

```
              className="h-4 w-4 text-amber-500 focus:ring-amber-400 border-white/30
rounded"
             />
             <label htmlFor="remember" className="ml-2 block text-sm text-gray-300">
              Remember me
             </label>
            </div>

            <button
             type="submit"
             disabled={isLoading}
             className="w-full bg-gradient-to-r from-amber-500 to-amber-600 text-indigo-900
py-3 px-4 rounded-lg font-medium hover:from-amber-400 hover:to-amber-500 transition-all
disabled:from-amber-400/70 disabled:to-amber-500/70 disabled:cursor-not-allowed
shadow-lg transform hover:-translate-y-1 uppercase tracking-wider"
            >
             {isLoading ? 'Signing in...' : 'Sign In'}
            </button>

            <div className="text-center mt-4">
             <p className="text-gray-300">
              Don't have an account?
              <Link to="/register" className="ml-1 text-amber-400 hover:text-amber-300 font-
medium">
               Sign up
              </Link>
             </p>
            </div>
           </form>
          </div>
         </div>
        </section>
       </>
      );
};

export default Login;

import { Link } from 'react-router-dom';
import Nav from '../components/Nav';

const NotFound = () =>
 { return (
   <>
    <Nav />
    <div className="min-h-screen bg-gradient-to-b from-indigo-50 to-purple-100 pt-32 pb-
12 px-4 flex flex-col items-center justify-center text-center">
```

```
      <div className="max-w-3xl mx-auto bg-white/80 backdrop-blur-sm rounded-2xl
shadow-xl p-8 border border-indigo-100">
        <div className="mb-8 flex justify-center">
          <div className="w-24 h-24 bg-indigo-100 rounded-full flex items-center justify-
center">
            <svg className="w-12 h-12 text-indigo-600" fill="none" stroke="currentColor"
viewBox="0 0 24 24" xmlns="http://www.w3.org/2000/svg">
              <path strokeLinecap="round" strokeLinejoin="round" strokeWidth={2} d="M9.172
16.172a4 4 0 015.656 0M9 10h.01M15 10h.01M21 12a9 9 0 11-18 0 9 9 0 0118 0z" />
            </svg>
          </div>
        </div>

        <h1 className="text-5xl font-serif font-bold text-gray-800 mb-4">404</h1>
        <h2 className="text-2xl font-bold text-gray-700 mb-6">Page Not Found</h2>
        <p className="text-gray-600 mb-8 text-lg">
          Oops! The page you're looking for seems to have checked out early.
        </p>

        <div className="flex flex-col sm:flex-row gap-4 justify-center">
          <Link
            to="/"
            className="bg-indigo-600 text-white px-6 py-3 rounded-lg font-medium hover:bg-
indigo-700 transition-colors shadow-md hover:shadow-lg"
          >
            Return Home
          </Link>
          <Link
            to="/booking"
            className="bg-amber-500 text-indigo-900 px-6 py-3 rounded-lg font-medium
hover:bg-amber-400 transition-colors shadow-md hover:shadow-lg"
          >
            Book a Room
          </Link>
        </div>
      </div>

      <div className="mt-12 text-gray-600">
        <p>Need assistance? <Link to="/contact" className="text-indigo-600 hover:text-
indigo-800 font-medium">Contact our support team</Link></p>
      </div>
    </div>
  </>
 );
};

export default NotFound;
```

```
import { useState } from 'react';
import { Link, useNavigate } from 'react-router-dom';
import Nav from "../components/Nav";
import { registerUser, loginUser } from '../services/authService';
import { toast, ToastContainer } from 'react-toastify';
import 'react-toastify/dist/ReactToastify.css';

const Register = () => {
  const navigate = useNavigate();
  const [isLoading, setIsLoading] = useState(false);
  const [formData, setFormData] =
  useState({ username: '',
    email: '',
    password: '',
    confirmPassword: ''
  });

  const handleChange = (e) =>
    { const { name, value } =
    e.target; setFormData(prev =>
    ({
      ...prev,
      [name]: value
    }));
  };

  const handleSubmit = async (e) => {
    e.preventDefault();

    // Basic validation
    if (formData.password !== formData.confirmPassword)
      { toast.error('Passwords do not match');
      return;
    }

    if (formData.password.length < 6) {
      toast.error('Password must be at least 6 characters long');
      return;
    }

    setIsLoading(true);

    try {
      // Remove confirmPassword before sending to API
      const { confirmPassword, ...registrationData } = formData;

      // Register the user
```

```
    await registerUser(registrationData);

    toast.success('Registration successful! Logging you in...');

    // Automatically log the user in
    await loginUser({
      email: formData.email,
      password: formData.password
    });

    // Redirect to homepage after registration and login
    setTimeout(() => {
      navigate('/');
    }, 1500);
  } catch (error) {
    console.error('Registration error:', error);
    toast.error(error.message || 'Registration failed. Please try again.');
  } finally {
    setIsLoading(false);
  }
 };

 return (
  <>
    <Nav />
    <ToastContainer position="top-right" autoClose={5000} />

    {/* Hero Section */}
    <section className="pt-32 pb-20 bg-indigo-50">
      <div className="container mx-auto px-4">
        <div className="max-w-3xl mx-auto text-center">
          <h1 className="text-4xl md:text-5xl font-serif font-bold text-gray-800 mb-6">Create
an Account</h1>
          <p className="text-xl text-gray-600 mb-8">Join Zorp Hotel to unlock exclusive
benefits and easily manage your bookings.</p>
        </div>
      </div>
    </section>
    {/* Registration Form Section */}
    <section className="py-16">
      <div className="container mx-auto px-4">
        <div className="max-w-md mx-auto bg-white rounded-lg shadow-md p-8">
          <form onSubmit={handleSubmit} className="space-y-6">
            <div>
              <label className="block text-gray-700 mb-2">Username</label>
              <input
                type="text"
```

```
              name="username"
              value={formData.username}
              onChange={handleChange}
              required
              className="w-full px-4 py-3 border border-gray-300 rounded-lg focus:ring
focus:ring-indigo-200 focus:border-indigo-600 outline-none transition"
              placeholder="Choose a username"
            />
          </div>

          <div>
            <label className="block text-gray-700 mb-2">Email Address</label>
            <input
              type="email"
              name="email"
              value={formData.email}
              onChange={handleChange}
              required
              className="w-full px-4 py-3 border border-gray-300 rounded-lg focus:ring
focus:ring-indigo-200 focus:border-indigo-600 outline-none transition"
              placeholder="Enter your email"
            />
          </div>

          <div>
            <label className="block text-gray-700 mb-2">Password</label>
            <input
              type="password"
              name="password"
              value={formData.password}
              onChange={handleChange}
              required
              className="w-full px-4 py-3 border border-gray-300 rounded-lg focus:ring
focus:ring-indigo-200 focus:border-indigo-600 outline-none transition"
              placeholder="Create a password"
            />
            <p className="text-sm text-gray-500 mt-1">Password must be at least 6
characters long</p>
          </div>

          <div>
            <label className="block text-gray-700 mb-2">Confirm Password</label>
            <input
              type="password"
              name="confirmPassword"
              value={formData.confirmPassword}
              onChange={handleChange}
```

```
              required
              className="w-full px-4 py-3 border border-gray-300 rounded-lg focus:ring
focus:ring-indigo-200 focus:border-indigo-600 outline-none transition"
              placeholder="Confirm your password"
             />
            </div>

            <div className="flex items-center">
             <input
              type="checkbox"
              id="terms"
              required
              className="h-4 w-4 text-indigo-600 focus:ring-indigo-500 border-gray-300
rounded"
             />
             <label htmlFor="terms" className="ml-2 block text-sm text-gray-700">
              I agree to the <a href="#" className="text-indigo-600 hover:text-indigo-
500">Terms and Conditions</a> and <a href="#" className="text-indigo-600 hover:text-
indigo-500">Privacy Policy</a>
             </label>
            </div>

            <button
             type="submit"
             disabled={isLoading}
             className="w-full bg-indigo-600 text-white py-3 px-4 rounded-lg font-medium
hover:bg-indigo-700 transition-colors disabled:bg-indigo-400 disabled:cursor-not-allowed"
            >
             {isLoading ? 'Creating Account...' : 'Create Account'}
            </button>

            <div className="text-center mt-4">
             <p className="text-gray-600">
              Already have an account?
              <Link to="/login" className="ml-1 text-indigo-600 hover:text-indigo-500 font-
medium">
               Sign in
              </Link>
             </p>
            </div>
           </form>
          </div>
         </div>
        </section>
       </>
      );
};
```

```
export default Register;
```

- components: Reusable UI components

```
 import { useState } from 'react';
import { changePassword } from '../services/authService';
import { toast } from 'react-toastify';

const ChangePasswordModal = ({ isOpen, onClose }) => {
  const [formData, setFormData] =
  useState({ currentPassword: '',
    newPassword: '',
    confirmPassword: ''
  });
  const [loading, setLoading] = useState(false);

  const handleChange = (e) =>
    { const { name, value } =
    e.target; setFormData(prev =>
    ({
      ...prev,
      [name]: value
    }));
  };

  const handleSubmit = async (e) => {
    e.preventDefault();

    // Validate passwords
    if (formData.newPassword !== formData.confirmPassword)
      { toast.error('New passwords do not match');
      return;
    }

    if (formData.newPassword.length < 6) {
      toast.error('New password must be at least 6 characters long');
      return;
    }

    setLoading(true);

    try {
      await changePassword({
        currentPassword: formData.currentPassword,
        newPassword: formData.newPassword
```

```
    });

    toast.success('Password changed successfully');

    // Reset form and close modal
    setFormData({ currentPasswo
    rd: '', newPassword: '',
    confirmPassword: ''
    });

    onClose();
   } catch (error) {
    console.error('Error changing password:', error);
    toast.error(error.message || 'Failed to change password');
   } finally {
    setLoading(false);
   }
  };

  if (!isOpen) return null;

  return (
   <div className="fixed inset-0 bg-black bg-opacity-50 flex items-center justify-center z-
50">
    <div className="bg-white rounded-lg p-8 max-w-md w-full">
     <div className="flex justify-between items-center mb-6">
      <h2 className="text-2xl font-bold text-gray-800">Change Password</h2>
      <button
       onClick={onClose}
       className="text-gray-500  hover:text-gray-700"
      >
       <svg className="w-6 h-6" fill="none" stroke="currentColor" viewBox="0 0 24 24">
        <path strokeLinecap="round" strokeLinejoin="round" strokeWidth={2} d="M6 18L18
6M6 6l12 12" />
       </svg>
      </button>
     </div>

     <form onSubmit={handleSubmit} className="space-y-4">
      <div>
       <label className="block text-gray-700 mb-2">Current Password</label>
       <input
        type="password"
        name="currentPassword"
        value={formData.currentPassword}
        onChange={handleChange}
```

```
            required
            className="w-full px-4 py-2 border border-gray-300 rounded-lg focus:ring
focus:ring-indigo-200 focus:border-indigo-600 outline-none transition"
          />
        </div>

        <div>
          <label className="block text-gray-700 mb-2">New Password</label>
          <input
            type="password"
            name="newPassword"
            value={formData.newPassword}
            onChange={handleChange}
            required
            className="w-full px-4 py-2 border border-gray-300 rounded-lg focus:ring
focus:ring-indigo-200 focus:border-indigo-600 outline-none transition"
          />
          <p className="text-sm text-gray-500 mt-1">Password must be at least 6 characters
long</p>
        </div>

        <div>
          <label className="block text-gray-700 mb-2">Confirm New Password</label>
          <input
            type="password"
            name="confirmPassword"
            value={formData.confirmPassword}
            onChange={handleChange}
            required
            className="w-full px-4 py-2 border border-gray-300 rounded-lg focus:ring
focus:ring-indigo-200 focus:border-indigo-600 outline-none transition"
          />
        </div>

        <div className="flex justify-end space-x-4 mt-6">
          <button
            type="button"
            onClick={onClose}
            className="px-4 py-2 border border-gray-300 rounded-md text-gray-700 hover:bg-
gray-50 transition-colors"
          >
            Cancel
          </button>
          <button
            type="submit"
            disabled={loading}
```

```
            className="px-4 py-2 bg-indigo-600 text-white rounded-md hover:bg-indigo-700
transition-colors disabled:bg-indigo-400 disabled:cursor-not-allowed"
          >
            {loading ? 'Changing...' : 'Change Password'}
          </button>
        </div>
      </form>
    </div>
  </div>
 );
};

export default ChangePasswordModal;

import { Link, useLocation } from 'react-router-dom';
import { useState, useEffect } from 'react';
import { isAuthenticated, logoutUser, getCurrentUser } from '../services/authService';

const Nav = ({ isDashboard = false }) => {
 const [isScrolled, setIsScrolled] = useState(false);
 const [isMobileMenuOpen, setIsMobileMenuOpen] = useState(false);
 const [isLoggedIn, setIsLoggedIn] = useState(false);
 const [user, setUser] = useState(null);
 const location = useLocation();

 // Check if current page is landing page
 const isLandingPage = location.pathname === '/';

 useEffect(() => {
  const handleScroll = () => {
   // Only apply scroll-based background on landing page
   if (isLandingPage) {
    setIsScrolled(window.scrollY > 10);
   } else {
    // Always show background on other pages
    setIsScrolled(true);
   }
  };

  const checkAuthStatus = () => {
   const authStatus = isAuthenticated();
   setIsLoggedIn(authStatus);
   if (authStatus) {
    setUser(getCurrentUser());
   }
  };
```

```
    // Set initial scroll state
    handleScroll();

    window.addEventListener('scroll', handleScroll);

    // Check auth status on mount and whenever localStorage changes
    checkAuthStatus();
    window.addEventListener('storage', checkAuthStatus);

    // Close mobile menu when route changes
    setIsMobileMenuOpen(false);

    return () => {
      window.removeEventListener('scroll', handleScroll);
      window.removeEventListener('storage', checkAuthStatus);
    };
  }, [location.pathname, isLandingPage]);

  const handleLogout = () => {
    logoutUser();
    setIsLoggedIn(false);
    setUser(null);
    setIsMobileMenuOpen(false);
    window.location.href = '/';
  };

  return (
    <nav className={`fixed w-full z-50 transition-all duration-300 ${isScrolled ? 'bg-gradient-
to-r from-purple-900 to-indigo-800 shadow-lg py-2' : 'bg-transparent py-4'}`}>
      <div className="container mx-auto px-4 flex justify-between items-center">
        <Link to="/" className="flex items-center">
          <span className={`font-serif text-2xl font-bold text-white ${isScrolled ? '' : 'drop-
shadow-md'}`}>
            Zorp <span className="text-amber-400">Hotel</span>
          </span>
        </Link>

        {/* Desktop Menu */}
        <div className="hidden md:flex items-center space-x-6">
          <Link to="/" className={`text-white hover:text-amber-300 font-medium transition-
colors text-sm uppercase tracking-wider`}>Home</Link>
          <Link to="/about" className={`text-white hover:text-amber-300 font-medium
transition-colors text-sm uppercase tracking-wider`}>About</Link>
          <Link to="/contact" className={`text-white hover:text-amber-300 font-medium
transition-colors text-sm uppercase tracking-wider`}>Contact</Link>

          {isLoggedIn ? (
```

```
          <>
            <Link to="/dashboard" className={`text-white hover:text-amber-300 font-medium
transition-colors text-sm uppercase tracking-wider`}>Dashboard</Link>
            {user?.role === 'admin' && (
              <Link to="/admin" className={`text-white hover:text-amber-300 font-medium
transition-colors text-sm uppercase tracking-wider`}>Admin Panel</Link>
            )}
            <button
              onClick={handleLogout}
              className="bg-amber-500 text-indigo-900 px-5 py-2 rounded-md hover:bg-amber-
400 transition-colors font-medium text-sm uppercase tracking-wider shadow-md
hover:shadow-lg"
            >
              Logout
            </button>
          </>
        ) : (
          <>
            <Link to="/login" className={`text-white hover:text-amber-300 font-medium
transition-colors text-sm uppercase tracking-wider`}>Login</Link>
            <Link to="/booking" className="bg-amber-500 text-indigo-900 px-5 py-2 rounded-
md hover:bg-amber-400 transition-colors font-medium text-sm uppercase tracking-wider
shadow-md hover:shadow-lg">Book Now</Link>
          </>
        )}
      </div>

      {/* Mobile menu button */}
      <button
        className="md:hidden  focus:outline-none"
        onClick={() => setIsMobileMenuOpen(!
        isMobileMenuOpen)} aria-label="Toggle mobile menu"
      >
        <svg className="w-6 h-6 text-white" fill="none" stroke="currentColor" viewBox="0 0
24 24" xmlns="http://www.w3.org/2000/svg">
          {isMobileMenuOpen ? (
            <path strokeLinecap="round" strokeLinejoin="round" strokeWidth={2} d="M6 18L18
6M6 6l12 12" />
          ) : (
            <path strokeLinecap="round" strokeLinejoin="round" strokeWidth={2} d="M4
6h16M4 12h16M4 18h16" />
          )}
        </svg>
      </button>
    </div>

    {/* Mobile Menu with improved animation */}
```

```
    <div
      className={`md:hidden fixed top-[60px] left-0 right-0 bg-gradient-to-r from-purple-900
to-indigo-800 shadow-lg transition-all duration-300 overflow-hidden z-40 ${
        isMobileMenuOpen ? 'max-h-96 opacity-100' : 'max-h-0 opacity-0'
      }`}
      aria-hidden={!isMobileMenuOpen}
    >
      <div className="px-4 py-2 space-y-3">
        <Link to="/" className="block text-white hover:text-amber-300 font-medium py-2
text-sm uppercase tracking-wider" onClick={() =>
setIsMobileMenuOpen(false)}>Home</Link>
        <Link to="/about" className="block text-white hover:text-amber-300 font-medium
py-2 text-sm uppercase tracking-wider" onClick={() =>
setIsMobileMenuOpen(false)}>About</Link>
        <Link to="/contact" className="block text-white hover:text-amber-300 font-medium
py-2 text-sm uppercase tracking-wider" onClick={() =>
setIsMobileMenuOpen(false)}>Contact</Link>

        {isLoggedIn ? (
          <>
            <Link to="/dashboard" className="block text-white hover:text-amber-300 font-
medium py-2 text-sm uppercase tracking-wider" onClick={() =>
setIsMobileMenuOpen(false)}>Dashboard</Link>
            {user?.role === 'admin' && (
              <Link to="/admin" className="block text-white hover:text-amber-300 font-
medium py-2 text-sm uppercase tracking-wider" onClick={() =>
setIsMobileMenuOpen(false)}>Admin Panel</Link>
            )}
            <button
              onClick={handleLogout}
              className="block w-full text-left text-white hover:text-amber-300 font-medium
py-2 text-sm uppercase tracking-wider"
            >
              Logout
            </button>
          </>
        ) : (
          <>
            <Link to="/login" className="block text-white hover:text-amber-300 font-medium
py-2 text-sm uppercase tracking-wider" onClick={() =>
setIsMobileMenuOpen(false)}>Login</Link>
            <Link to="/booking" className="block bg-amber-500 text-indigo-900 px-4 py-2
rounded-md hover:bg-amber-400 text-center font-medium my-2 text-sm uppercase
tracking-wider shadow-md" onClick={() => setIsMobileMenuOpen(false)}>Book
Now</Link>
          </>
        )}
      </div>
```

```
      </div>
    </nav>
  );
};

export default Nav;

   - services: API service calls

   import api from './api';

export const getAllBookings = async () =>
  { try {
    const response = await api.get('/admin/bookings');
    return response.data;
  } catch (error) {
    throw error.response?.data || { message: 'Failed to fetch bookings' };
  }
};

export const getAllUsers = async () =>
  { try {
    const response = await api.get('/admin/users');
    return response.data;
  } catch (error) {
    throw error.response?.data || { message: 'Failed to fetch users' };
  }
};

export const createRoom = async (roomData) =>
  { try {
    // Format room data to match the backend schema
    const formattedData = {
      ...roomData,
      amenities: Array.isArray(roomData.amenities)
        ? roomData.amenities
        : roomData.amenities?.split(',').map(item => item.trim()).filter(Boolean) || [],
      images: Array.isArray(roomData.images)
        ? roomData.images
        : roomData.images?.split(',').map(item => item.trim()).filter(Boolean) || []
    };

    const response = await api.post('/admin/rooms', formattedData);
    return response.data;
  } catch (error) {
    throw error.response?.data || { message: 'Failed to create room' };
  }
```

```
};

export const updateRoom = async (roomId, roomData) => {
  try {
    // Format room data to match the backend schema
    const formattedData = {
      ...roomData,
      amenities: Array.isArray(roomData.amenities)
        ? roomData.amenities
        : roomData.amenities?.split(',').map(item => item.trim()).filter(Boolean) || [],
      images: Array.isArray(roomData.images)
        ? roomData.images
        : roomData.images?.split(',').map(item => item.trim()).filter(Boolean) || []
    };

    const response = await api.put(`/admin/rooms/${roomId}`, formattedData);
    return response.data;
  } catch (error) {
    throw error.response?.data || { message: 'Failed to update room' };
  }
};

export const deleteRoom = async (roomId) =>
  { try {
    const response = await api.delete(`/admin/rooms/${roomId}`);
    return response.data;
  } catch (error) {
    throw error.response?.data || { message: 'Failed to delete room' };
  }
};

export const updateUser = async (userId, userData) =>
  { try {
    const response = await api.put(`/admin/users/${userId}`, userData);
    return response.data;
  } catch (error) {
    throw error.response?.data || { message: 'Failed to update user' };
  }
};

export const deleteUser = async (userId) =>
  { try {
    const response = await api.delete(`/admin/users/$
    {userId}`); return response.data;
  } catch (error) {
    throw error.response?.data || { message: 'Failed to delete user' };
  }
```

```
};

export const updateBooking = async (bookingId, bookingData) =>
 { try {
   const response = await api.put(`/admin/bookings/${bookingId}`, bookingData);
   return response.data;
 } catch (error) {
   throw error.response?.data || { message: 'Failed to update booking' };
 }
};

export const getAllRooms = async () =>
 { try {
   const response = await api.get('/admin/rooms');
   return response.data;
 } catch (error) {
   throw error.response?.data || { message: 'Failed to fetch rooms' };
 }
};

import axios from 'axios';
import { toast } from 'react-toastify';

// Create axios instance with base URL
const baseURL = import.meta.env.VITE_API_URL || 'http://localhost:3000';

// Remove /api prefix if using the deployed backend
const apiPrefix = baseURL.includes('vercel.app') ? '' : '/api';

const api = axios.create({
 baseURL: `${baseURL}${apiPrefix}`,
 headers: {
   'Content-Type': 'application/json',
 },
 // Disable credentials for CORS
 withCredentials: false
});

// Request interceptor to add auth token to requests
api.interceptors.request.use(
 (config) => {
   const token = localStorage.getItem('token');
   if (token) {
     config.headers.Authorization = `Bearer ${token}`;
   }

   // Log request for debugging
```

```
    console.log('API Request:', config.method.toUpperCase(), config.url);

    return config;
  },
  (error) => {
    console.error('Request Error:', error);
    return Promise.reject(error);
  }
);

// Response interceptor to handle errors
api.interceptors.response.use(  (respons
e) => response,
  (error) => {
    // Create a standardized error object
    const errorResponse = {
      status: error.response?.status || 500,
      message: error.response?.data?.message || 'An unexpected error occurred',
      errors: error.response?.data?.errors || []
    };

    // Log error for debugging
    console.error('API Error:', errorResponse);

    // Handle CORS errors
    if (error.message === 'Network Error')
      { console.error('Network Error Details:', error);
      toast.error('Network error: Could not connect to the server. This might be a CORS
issue.');
      return Promise.reject({ message: 'Network error: Could not connect to the server' });
    }

    // Handle specific error codes
    if (errorResponse.status === 401) {
      // Unauthorized - clear local storage and redirect to login
      localStorage.removeItem('token');
      localStorage.removeItem('user');
      window.location.href = '/login';
    } else if (errorResponse.status === 403) {
      // Forbidden - redirect to home or show permission error
      toast.error('You do not have permission to perform this action');
    } else if (errorResponse.status === 404) {
      // Not found
      toast.error('The requested resource was not found');
    } else if (errorResponse.status === 422 || errorResponse.status === 400) {
      // Validation errors - show detailed validation errors
      if (errorResponse.errors.length > 0) {
```

```
        errorResponse.errors.forEach(err =>
          { toast.error(err.msg || err.message);
        });
      } else {
        toast.error(errorResponse.message);
      }
    } else {
      // Generic error message
      toast.error(errorResponse.message);
    }

    return Promise.reject(errorResponse);
  }
);

export default api;


import api from './api';

export const registerUser = async (userData) =>
  { try {
    const response = await api.post('/auth/register', userData);
    return response.data;
  } catch (error) {
    throw error.response?.data || { message: 'Registration failed' };
  }
};

export const loginUser = async (credentials) =>
  { try {
    const response = await api.post('/auth/login', credentials);
    const { token } = response.data;

    // Store token in localStorage
    localStorage.setItem('token', token);

    // Get user profile and store in localStorage
    const userProfile = await getUserProfile();
    localStorage.setItem('user', JSON.stringify(userProfile));

    return userProfile;
  } catch (error) {
    throw error.response?.data || { message: 'Login failed' };
  }
};
```

```
export const logoutUser = () => {
  localStorage.removeItem('token');
  localStorage.removeItem('user');
};

export const getUserProfile = async () =>
  { try {
    const response = await api.get('/auth/profile');
    return response.data;
  } catch (error) {
    throw error.response?.data || { message: 'Failed to get user profile' };
  }
};

export const changePassword = async (passwordData) =>
  { try {
    const response = await api.post('/auth/change-password', passwordData);
    return response.data;
  } catch (error) {
    throw error.response?.data || { message: 'Failed to change password' };
  }
};

export const isAuthenticated = () => {
  return localStorage.getItem('token') !== null;
};

export const getCurrentUser = () =>
  { const user =
  localStorage.getItem('user'); return
  user ? JSON.parse(user) : null;
};

import api from './api';

export const createBooking = async (bookingData) =>
  { try {
    const response = await api.post('/bookings', bookingData);
    return response.data;
  } catch (error) {
    throw error.response?.data || { message: 'Booking failed' };
  }
};

export const getBookingHistory = async () =>
  { try {
    const response = await api.get('/bookings/history');
    return response.data;
```

```
  } catch (error) {
    throw error.response?.data || { message: 'Failed to fetch booking history' };
  }
};

export const cancelBooking = async (bookingId) =>
  { try {
    const response = await api.delete(`/bookings/${bookingId}`);
    return response.data;
  } catch (error) {
    throw error.response?.data || { message: 'Failed to cancel booking' };
  }
};

import api from './api';

export const getAllRooms = async () => {
  try {
    const response = await api.get('/rooms');
    return response.data;
  } catch (error) {
    throw error.response?.data || { message: 'Failed to fetch rooms' };
  }
};

export const searchRooms = async (queryParams) =>
  { try {
    const response = await api.get('/rooms/search', { params: queryParams });
    return response.data;
  } catch (error) {
    throw error.response?.data || { message: 'Failed to search rooms' };
  }
};

export const getFeaturedRooms = async () => {
  try {
    const response = await api.get('/rooms/featured');
    return response.data;
  } catch (error) {
    throw error.response?.data || { message: 'Failed to fetch featured rooms' };
  }
};
export const getRoomById = async (roomId) =>
  { try {
    const response = await api.get(`/rooms/${roomId}`);
    return response.data;
  } catch (error) {
```

```
    throw error.response?.data || { message: 'Failed to fetch room details' };
  }
};
```

SCREEN LAYOUTS

10. SCREEN LAYOUTS







Our History

Founded in 2005, Zorp Hotel began as a small boutique establishment with a clear vision: to provide an unparalleled hospitality experience that combines luxury with authentic personalized service.

Over the years, we've grown to become one of the most prestigious hotels in the region, known for our attention to detail and commitment to guest satisfaction.




Zorp Hotel
HOME
ABOUT
CONTACT
LOGIN
BOOK NOW
Book Your Perfect Stay
Select from our premium rooms and suites for an unforgettable experience at our luxury hotel.
Select Your Stay Details
Select Dates
Check-in Date *
dd-mm-yyyy
Check-out Date *
dd-mm-yyyy
Booking Summary
Please select a room to continue
Room Charge
Rs.0
Taxes & Fees
Rs.0

FUTURE ENHANCEMENTS

11. FUTURE ENHANCEMENT

Potential future enhancements include:

- Integration with payment gateways for bookings

- Advanced analytics and reporting

- Mobile application development

CONCLUSIONS

12. CONCLUSIONS

The Hotel Management application demonstrates the potential of technology in optimizing hotel operations. By providing a comprehensive platform for managing bookings and resources, it enhances service delivery and customer satisfaction.

The Hotel Management System (HMS) project has successfully designed and developed a comprehensive software application to manage the day-to-day operations of a hotel. The system provides a user-friendly interface for hotel staff to manage rooms, reservations, guest information,services,andbilling.

BIBILIOGRAPHY

13. BIBILIOGRAPHY

-Node.js Documentatiohttps://nodejs.org/en/docs/

-React Documentation: https://reactjs.org/docs/

-MongoDB Documentation: https://docs.mongodb.com/